A CONSTRUCTIVE PROOF OF RICE'S THEOREM AND THE HALTING PROBLEM VIA HILBERT'S TENTH PROBLEM

JONATHAN BROSSARD

e-mail address: brossardj@acm.org

Abstract. Rice's theorem states that no non-trivial semantic property of programs is decidable. Classical proofs proceed by reduction from the halting problem, invoking the law of excluded middle (LEM) twice: once through diagonalization, and once through a case split on whether the always-diverging program $\bot$ satisfies the property in question. We present a proof that is *constructive relative to the undecidability of Hilbert's Tenth Problem* (MRDP): valid in intuitionistic logic, requiring neither diagonalization nor self-reference, and adding no classical reasoning beyond the MRDP assumption itself.

The key idea is a two-witness construction. Given a non-trivial property P, we attach to each Diophantine polynomial D a pair of programs S_D^0, S_D^1 that behave like the negative and positive witnesses for P when D is solvable, and both diverge identically when it is not. A hypothetical decider for P would therefore decide Diophantine solvability via the difference $\delta_D = \text{Decide}_P(S_D^1) - \text{Decide}_P(S_D^0)$ — contradicting the MRDP theorem. The argument is structured as two separate implications, never asserting a disjunction about solvability, and never examining $P(\bot)$. The undecidability of the halting problem follows as an immediate corollary: a single application of Rice's theorem to the `Terminates` property.

A formalization in the Rocq proof assistant[1] confirms both results within a step-indexed model of computation, with the undecidability of Hilbert's Tenth Problem as the sole external axiom. Both `Rice_Theorem` and `Halting_Problem` are *closed under the global context*[2].

1. Introduction

Rice's theorem [Ric53] states that no non-trivial semantic property of programs is decidable. We prove it constructively *relative to MRDP undecidability*: our proof is valid in intuitionistic logic and adds no classical reasoning beyond the undecidability of Hilbert's Tenth Problem. The halting problem [Tur36] is the most famous specific instance: no algorithm decides, for an arbitrary program and input, whether the program halts. Standard textbook proofs of both results invoke the law of excluded middle (LEM) in essential ways. For Rice's theorem, LEM appears in at least two places: through diagonalization, which requires asserting that a hypothetically constructed program either halts or diverges; and through a case split on whether the always-diverging program $\bot$ satisfies the property P in question — the so-called $P(\bot)$ split. For the halting problem, LEM enters through self-reference: the diagonal program halts if and only if the hypothetical decider says it does

Key words and phrases: Rice's theorem, halting problem, constructive logic, intuitionistic logic, MRDP theorem, Hilbert's tenth problem, Rocq, step-indexed semantics, undecidability, formal verification, diagonalization-free.

© CONSTRUCTIVE RICE AND HALTING VIA MRDP
Creative Commons

not. Our proof eliminates all of these uses of LEM simultaneously, for both results and without any self-reference, by reducing directly to Hilbert's Tenth Problem rather than to the halting problem.

Why a constructive proof matters. Constructive proofs of undecidability results carry computational content that classical proofs lack. In intuitionistic type theory and in systems such as Coq and Agda that are used for program extraction, a classical proof cannot be directly executed or extracted into a verified program. A constructive proof of Rice's theorem would, in principle, allow the undecidability argument to be internalized in such systems without axioms beyond the undecidability of Hilbert's Tenth Problem. It also contributes to constructive reverse mathematics, where the goal is to identify precisely which non-constructive principles are required for each theorem. Furthermore, this proof can be seen as a *non-branching reduction*: information is encoded in the relational difference δ_D between two programs rather than in a case distinction on a single program, a structural property that may be of independent interest.

This paper. We present a proof of Rice's theorem that is valid in intuitionistic logic, eliminating both classical steps. The key insight is to reduce to Hilbert's Tenth Problem rather than to the halting problem directly. Given a non-trivial semantic property P with witnesses e_0 (failing P) and e_1 (satisfying P), we construct for each Diophantine polynomial D a pair of programs S_D^0 and S_D^1 that behave like e_0 and e_1 respectively when D has a solution, and both diverge identically when it does not. A hypothetical decider Decide_P would therefore distinguish the two cases via the difference $\delta_D = \mathrm{Decide}_P(S_D^1) - \mathrm{Decide}_P(S_D^0)$, deciding Diophantine solvability — which is impossible by the MRDP theorem. The argument is structured as two separate implications (solvable $\Rightarrow \delta_D = 1$; unsolvable $\Rightarrow \delta_D = 0$), never asserting the disjunction "D solvable or unsolvable," and never examining $P(\bot)$.

Formalization. A formalization in the Rocq proof assistant accompanies the paper. The formalization uses a step-indexed model of computation. `Rice_Theorem` and `Halting_Problem` are proved without any classical axioms. Every theorem in the development — including `MRDP_from_LW` — uses zero classical axioms. The arithmetic encoding `poly_encode_correct` is fully proved (zero admitted goals, zero classical axioms). The sole axiom is `H10C_SAT_undec` (the MRDP undecidability result), a direct import from `coq-library-undecidability` [LWF22]; `LW_H10c_undec` is derived from it constructively. No totality condition is imposed on the witnesses: the key design choice is that the witness programs S_D^b pass their full fuel budget to e_b unchanged, so the fuel-offset problem that previously required totality does not arise.

Novelty. The closest prior work is Forster, Kirst, and Smolka [FKS19], who formalize Rice's theorem in Coq. Their proof follows the classical Rogers presentation: for each program e they construct a new program h_e that simulates e and then behaves like a positive witness if e halts. This requires two classical steps: (1) the s-1-1 theorem (via the Recursion Theorem, a diagonalization) to make h_e computable from e; and (2) a case split on $P(\bot) \vee \neg P(\bot)$ to decide the direction of the reduction. Their formalization uses the `Classical` library throughout.

Our proof avoids both steps structurally. The witness programs S_D^b are constructed from a Diophantine polynomial and a bit — no self-reference, no fixed-point. The $P(\bot)$ split is irrelevant because when D has no solution, $S_D^0 \equiv S_D^1 \equiv \bot$ and extensionality gives

$P(S^0_D) \Leftrightarrow P(S^1_D)$ with no knowledge of $P(\bot)$. The Rocq formalization is fully intuitionistic; no classical library is imported.

Constructive audit. Table 1 summarises the axiom dependencies of every theorem in the development. The sole external axiom is `H10C_SAT_undec`, proved by Larchey-Wendling and Forster [LWF22] in CIC without classical axioms; our development adds no further reasoning of any kind. `Rice_Theorem` and `Halting_Problem` carry zero classical axioms (`Print Assumptions` returns *closed under the global context*).

Structure. Section 2 establishes the step-indexed model (Proposition 2.2 justifies its faithfulness), and defines observational equivalence and semantic properties. Section 3 recalls the MRDP theorem and its constructive status. Section 4 defines the two-witness construction. Section 5 proves the main theorem. Section 6 derives the undecidability of the halting problem as a corollary (one line of Rocq). Section 7 discusses the decidability frontier for bounded programs and states an open problem. Section 8 surveys related work. The appendix contains the complete Rocq formalization.

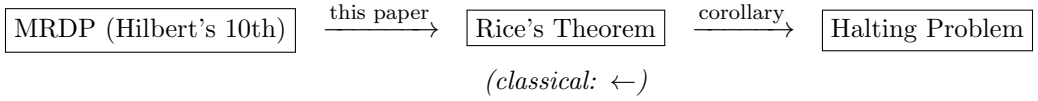


Figure 1: Reduction directions. Classical proofs derive Rice's theorem *from* the halting problem (left arrow). This paper reduces *from* MRDP (right arrows), with the halting problem as a corollary.

2. Preliminaries

2.1. **Partial Functions and Convergence.** We work in a standard model of partial computable functions. For a program e and input x, we write $\varphi_e(x) \downarrow$ if the computation converges, and $\varphi_e(x) \uparrow$ if it diverges. We denote by $\bot$ the specific program that diverges on every input, i.e., $\forall x : \varphi_\bot(x) \uparrow$. Note that $\bot$ here denotes a concrete program (e.g., the program that runs an infinite loop on every input), not a logical proposition; we use $\bot$ exclusively in this sense throughout.

2.2. **Constructive Equality of Partial Functions.** A central concern in constructive logic is the meaning of equality for partial functions. We adopt a step-indexed notion suitable for formalization.

Definition 2.1 (Observational Equivalence)**.** For a step-indexed model where each program is a function $e : \mathbb{N} \to (\mathbb{N} \to \text{option } \mathbb{N})$ (input x then fuel k), two programs e and f are *observationally equivalent*, written $\varphi_e \simeq \varphi_f$, if for every input x there exists a fuel bound N such that for all $k \geq N$, $e(x, k) = f(x, k)$.

This definition is strictly constructive: it does not assert $\varphi_e(x) \downarrow \vee \varphi_e(x) \uparrow$ for any particular x; it requires eventual agreement in the step-indexed model.

Proposition 2.2 (Faithfulness of the step-indexed model)**.** This is a paper-level metatheoretic argument connecting the step-indexed model to classical computability theory. It is not formalized in `rice.v` and the main theorems do not depend on it — see Remark 2.3.

Let $\varphi_e : \mathbb{N} \rightharpoonup \mathbb{N}$ denote the partial function computed by program e in the standard sense (Turing machines, or any equivalent model). Define the step-indexed lifting $\hat{e}(x,k) = \mathsf{Some}\, v$ if e on input x terminates in at most k steps with output v, and None otherwise. Then:

(1) (Monotonicity) *$\hat{e}$ is stable in the sense of Definition 2.1: if $\hat{e}(x,k) = \mathsf{Some}\, v$ then $\hat{e}(x,k') = \mathsf{Some}\, v$ for all $k' \geq k$.*

(2) (Soundness) *If $\hat{e} \simeq \hat{f}$ then $\varphi_e = \varphi_f$ as partial functions: for all x, $\varphi_e(x) \downarrow v$ iff $\varphi_f(x) \downarrow v$.*

(3) (Completeness) *If $\varphi_e = \varphi_f$ then $\hat{e} \simeq \hat{f}$.*

Proof. (1) Immediate from the definition of $\hat{e}$: termination in k steps implies termination in $k' \geq k$ steps with the same output. (2) Suppose $\hat{e} \simeq \hat{f}$ and $\varphi_e(x) \downarrow v$. Then $\hat{e}(x,k) = \mathsf{Some}\, v$ for some k. By Definition 2.1, there exists N such that $\hat{e}(x,k) = \hat{f}(x,k)$ for all $k \geq N$. Taking $k = \max(k_0, N)$ where k_0 is the termination step gives $\hat{f}(x,k) = \mathsf{Some}\, v$, so $\varphi_f(x) \downarrow v$. By symmetry, $\varphi_e = \varphi_f$. (3) Suppose $\varphi_e = \varphi_f$. For any x, if $\varphi_e(x) \uparrow$ then $\hat{e}(x,k) = \mathsf{None} = \hat{f}(x,k)$ for all k, so agreement holds trivially with $N = 0$. If $\varphi_e(x) \downarrow v$, let k_e (resp. k_f) be the number of steps for e (resp. f) to terminate on x. Then $\hat{e}(x,k) = \mathsf{Some}\, v$ for all $k \geq k_e$ and $\hat{f}(x,k) = \mathsf{Some}\, v$ for all $k \geq k_f$; taking $N = \max(k_e, k_f)$ gives agreement. □

Remark 2.3 (Scope of the formalization)**.** Proposition 2.2 is a *metatheoretic* statement connecting the step-indexed model to classical computability theory. The Rocq formalization does *not* depend on it: `Rice_Theorem` and `Halting_Problem` are proved entirely within the step-indexed model, with no reference to Turing machines or partial recursive functions. What the formalization proves is: *within any model of computation satisfying the step-indexed interface (Definition 2.1), no non-trivial semantic property is decidable.* Proposition 2.2 then confirms that the standard Turing model instantiates this interface, so the result transfers. The Rocq formalization takes monotonicity (part 1) as a definition; parts 2 and 3 are paper-level arguments that would require a concrete computation model to mechanize fully [AM01].

Remark 2.4. Observational equivalence is a relation, not a decision procedure. It serves as a definition of extensional equality in the step-indexed setting. For programs that converge, it is consistent with Kleene equality: if $\varphi_e(x) \downarrow v$ and $\varphi_f(x) \downarrow v'$ for all x, then $\varphi_e \simeq \varphi_f$ implies $v = v'$. Divergence is handled by requiring eventual agreement of the step-indexed outputs rather than asserting a disjunction on convergence.

2.3. **Semantic Properties.**

Definition 2.5 (Semantic Property)**.** A predicate P on programs is a *semantic property* (or *extensional property*) if it respects observational equivalence:

$$\forall e, f \,:\, \varphi_e \simeq \varphi_f \;\Rightarrow\; \big(P(e) \Leftrightarrow P(f)\big).$$

Definition 2.6 (Constructive Non-Triviality)**.** A semantic property P is *non-trivial* if there exist explicitly given programs e_0 and e_1 such that:

$$\neg P(e_0) \qquad \text{and} \qquad P(e_1).$$

In intuitionistic logic, mere existence is insufficient; effective witnesses must be provided. No totality condition is imposed on the witnesses; the proof is valid for arbitrary partial programs e_0 and e_1.

Remark 2.7. We make no assumption about $P(\bot)$. In particular, we do not assume $P(\bot) \vee \neg P(\bot)$, which would invoke LEM. The entire proof avoids determining the truth value of $P(\bot)$.

Table 1 summarises the axiom dependencies of every theorem in the development; readers may wish to consult it before the proof.

Theorem	**Axioms**	**Classical?**
`poly_encode_correct`	none	no
`LW_H10c_undec`	`H10C_SAT_undec`	no
`MRDP_from_LW`	`H10C_SAT_undec`	no
`Rice_Theorem`	closed	no
`Halting_Problem`	closed	no
`H10C_SAT_undec` (in LW)	FRACTRAN→Minsky→Halting	no — proved in CIC

Table 1: Axiom provenance. "Closed" means `Print Assumptions` returns *Closed under the global context*. `H10C_SAT_undec` is proved by Larchey-Wendling and Forster in CIC without classical axioms [LWF22]; the proof encodes register-machine transitions as Diophantine constraints via an explicit construction that requires no LEM.

3. The MRDP Theorem and its Constructive Status

The MRDP theorem (Davis–Putnam–Robinson–Matiyasevich, 1970) states that every recursively enumerable set is Diophantine: for any recursively enumerable set $S \subseteq \mathbb{N}$, there exists a polynomial $D_S(\vec{x}, y)$ with integer coefficients such that:

$$y \in S \iff \exists \vec{x} \in \mathbb{N}^k \,:\, D_S(\vec{x}, y) = 0.$$

The construction of D_S from a program enumerating S is algorithmic and explicit. No proof by contradiction or diagonalization is required; the theorem reduces the membership problem to polynomial arithmetic through a finite, computable sequence of steps.

Remark 3.1 (Constructive status of MRDP)**.** The polynomial construction in MRDP uses induction and case analysis but no LEM on undecidable propositions; the genuinely classical step is the undecidability argument, not the polynomial encoding. Ruitenburg [Rui85] established the forward direction (solvability from a halting derivation) in intuitionistic arithmetic with countable choice. For our proof, MRDP enters exactly once via the axiom `H10C_SAT_undec`, proved by Larchey-Wendling and Forster [LWF22] in CIC without classical logic.

In particular, let $K = \{e \mid \varphi_e(e)\downarrow\}$ be the halting set. K is recursively enumerable but not computable. MRDP yields an explicit polynomial $D_K(\vec{x}, e)$ such that:

$$\varphi_e(e)\downarrow \iff \exists \vec{x} \in \mathbb{N}^k \,:\, D_K(\vec{x}, e) = 0.$$

The construction of D_K from a Turing machine description is algorithmic: register-machine transitions are encoded as Diophantine constraints, and their conjunction is folded into a

single polynomial via the sum-of-squares identity ($c_1 = 0 \wedge c_2 = 0 \iff c_1^2 + c_2^2 = 0$). The axiom `H10C_SAT_undec` packages this result: there is no computable boolean separator for Diophantine solvability, which is exactly what is needed to drive the contradiction in Theorem 5.1.

Corollary 3.2 (MRDP, applied)**.** *There is no total computable function F with codomain $\{0,1\}$ satisfying:*

$$(D \text{ solvable} \Rightarrow F(D) = 1) \quad \text{and} \quad (D \text{ unsolvable} \Rightarrow F(D) = 0).$$

Remark 3.3 (Constructive status of Corollary 3.2)**.** The undecidability of K is a negative statement ($\neg\exists f$, f decides K), proved constructively by deriving $\bot$ from the assumption of a decider ($\neg P \equiv P \to \bot$). No positive disjunction about halting is asserted.

4. The Two-Witness Construction

Let P be a non-trivial semantic property with witnesses e_0 (satisfying $\neg P(e_0)$) and e_1 (satisfying $P(e_1)$), as given by Definition 2.6.

The following example illustrates the key behaviour before the formal definition.

Example 4.1 (Concrete instantiation)**.** Let $P =$ `Terminates`, $e_0 =$ `diverge`, $e_1 =$ `halt`, and $D(x, y, z) = x^2 - y^2 - z^2 - 1$. The programs S_D^0 and S_D^1 both enumerate triples $(x, y, z) \in \mathbb{N}^3$ checking whether D has a solution.

- *If a solution exists* (it does: $x = 1, y = 0, z = 0$): at step m where a solution is first found, S_D^0 switches to behaving like `diverge` (never halts), while S_D^1 switches to `halt` (always halts). Thus $P(S_D^1) =$ `true`, $P(S_D^0) =$ `false`, and $\delta_D = 1$.
- *If no solution existed*: both programs would diverge on every input ($\varphi_{S_D^0} \simeq \varphi_{S_D^1} \simeq \varphi_\bot$), so any decider `dec` would return the same value for both, giving $\delta_D = 0$ — regardless of $P(\bot)$.

The key point: no case split on $P(\bot)$ is needed. When D is unsolvable, both witnesses are $\varphi_\bot$, and their identical behavior forces $\delta_D = 0$ without asking whether $\bot$ terminates. The same pattern works for any non-trivial P and any D.

For any polynomial $D(\vec{x})$ with integer coefficients and k variables, we define two programs S_D^0 and S_D^1 as follows.

Definition 4.2 (Witness Programs)**.** For $b \in \{0, 1\}$, the program S_D^b operates on any input y as follows:

(1) Enumerate all tuples $\vec{x} \in \mathbb{N}^k$ in a canonical order (e.g., by increasing ℓ_1-norm, with ties broken lexicographically).
(2) For each tuple $\vec{x}$, compute $D(\vec{x})$ (a finite integer computation).
(3) If $D(\vec{x}) = 0$ for some tuple reached at step n: on every call $S_D^b(y, \mathit{fuel})$ with $\mathit{fuel} \geq n$, return $e_b(y, \mathit{fuel})$ directly. Each call is self-contained; S_D^b inspects *fuel* to determine whether a solution would have been found by that step, with no shared mutable state.
(4) If the enumeration continues indefinitely (i.e., no solution is found): S_D^b diverges on every input.

Remark 4.3. The construction of S_D^b from D, k, and b is entirely explicit and computable. No self-reference, fixed point, or diagonalization is involved.

Lemma 4.4 (Behavior when solvable)**.** *If* $\exists \vec{x} \in \mathbb{N}^k \,:\, D(\vec{x}) = 0$*, then* $\varphi_{S_D^b} \simeq \varphi_{e_b}$ *for each* $b \in \{0, 1\}$*.*

Proof. Suppose $D(\vec{x}_0) = 0$ for some $\vec{x}_0$. Since the enumeration is exhaustive and canonical, S_D^b will encounter $\vec{x}_0$ at some finite step n. After this step, S_D^b passes its full fuel budget to e_b unchanged (step 3 of Definition 4.2), so for all fuel $\geq n$ we have $S_D^b(y, \texttt{fuel}) = e_b(y, \texttt{fuel})$ exactly. Observational equivalence $\varphi_{S_D^b} \simeq \varphi_{e_b}$ then holds with threshold $N = n$, with no totality condition required on e_b. □

Lemma 4.5 (Behavior when unsolvable)**.** *If* $\forall \vec{x} \in \mathbb{N}^k \,:\, D(\vec{x}) \neq 0$*, then* $\varphi_{S_D^0} \simeq \varphi_\perp$ *and* $\varphi_{S_D^1} \simeq \varphi_\perp$*.*

Proof. If D has no solution, step 3 is never reached, so S_D^b diverges on every input. By Definition 2.1, $\varphi_{S_D^b} \simeq \varphi_\perp$. □

Corollary 4.6 (Joint behavior)**.** *If* D *is solvable:* $\varphi_{S_D^1} \simeq \varphi_{e_1}$ *and* $\varphi_{S_D^0} \simeq \varphi_{e_0}$*. If* D *is unsolvable:* $\varphi_{S_D^1} \simeq \varphi_{S_D^0}$ *(both equivalent to* $\varphi_\perp$*).*

5. The Main Theorem

Our decider returns values in $\mathbb{Z}$ rather than $\mathbb{N}$, so that the separation value $\delta_D = \text{Decide}_P(S_D^1) - \text{Decide}_P(S_D^0)$ is computed over the integers: $\delta_D = 0$ when D is unsolvable and $\delta_D = 1$ when D is solvable, and these two values are unambiguously distinct regardless of sign. In the Rocq formalization this choice also avoids natural-number subtraction truncation ($n - m = 0$ when $n < m$ in `nat`), which would otherwise collapse a signed difference to 0. The lemma `Z_sep_to_bool_sep` (§10) confirms that any $\mathbb{Z}$-valued separator immediately yields a boolean one, so the choice of codomain is not restrictive.

In the Rocq development, `MRDP_undecidable` appears as a `Hypothesis` inside `Section Rice_Theorem` — standard Rocq idiom meaning that `Rice_Theorem` and `Halting_Problem` are universally quantified over it at section close. `Theorem MRDP_from_LW` (Appendix A, §6.3) discharges this hypothesis, so `Print Assumptions` returns *Closed under the global context* for both theorems.

Theorem 5.1 (Rice's Theorem, Constructive Version)**.** *Let* P *be a non-trivial semantic property in the sense of Definition 2.6. Then there is no program* Decide_P *that decides* P*: for every program* e*,* $\text{Decide}_P(e)$ *terminates and returns* 1 *if* $P(e)$ *holds and* 0 *if* $\neg P(e)$ *holds.*

Remark 5.2 (Totality of the constructed function)**.** Since Decide_P is assumed to terminate on every input and return only 0 or 1, we have $a_D, b_D \in \{0, 1\}$ for all D. Consequently $\delta_D \in \{-1, 0, 1\}$ in general, but Lemmas A and B together imply $\delta_D \in \{0, 1\}$ under the assumption of a correct decider. Moreover, since Decide_P terminates on every program, the constructed function $F := \lambda(ar, D).\, \delta_{ar,D}$ is total, as required by Corollary 3.2.

Remark 5.3 (Constructive validity of the two implications)**.** Lemmas A and B are proved as separate implications; we never assert the disjunction "D solvable or unsolvable." The contradiction is reached by constructing a total function satisfying Corollary 3.2.

Proof. Assume for contradiction that Decide_P exists, with codomain $\{0,1\}$, satisfying $\mathrm{Decide}_P(e) = 1 \Leftrightarrow P(e)$. Define:

$$a_D \;=\; \mathrm{Decide}_P(S^1_D), \qquad b_D \;=\; \mathrm{Decide}_P(S^0_D), \qquad \delta_D \;=\; a_D - b_D.$$

Both a_D and b_D are computable from D, hence so is δ_D.

Lemma A: If D is solvable, then $\delta_D = 1$.

By Lemma 4.4 and extensionality, $P(S^1_D) \Leftrightarrow P(e_1)$ and $P(S^0_D) \Leftrightarrow P(e_0)$. Since $P(e_1)$ holds, $a_D = 1$; since $\neg P(e_0)$ holds, $b_D = 0$. Thus $\delta_D = 1$.

Lemma B: If D is unsolvable, then $\delta_D = 0$.

By Lemma 4.5, $\varphi_{S^1_D} \simeq \varphi_{S^0_D}$. By extensionality, $P(S^1_D) \Leftrightarrow P(S^0_D)$, so $a_D = b_D$ and $\delta_D = 0$. The truth value of $P(\bot)$ is never needed.

Contradiction. Lemma A and Lemma B together state:

$$(D \text{ solvable } \Rightarrow\; \delta_D = 1) \qquad \text{and} \qquad (D \text{ unsolvable } \Rightarrow\; \delta_D = 0).$$

Both implications hold intuitionistically. The function $D \mapsto \delta_D$ is therefore computable, has codomain $\{0,1\}$ under our assumption, and satisfies the conditions of Corollary 3.2, which asserts no such function exists. Contradiction.

Therefore no decider for P exists. □ □

6. Corollary: The Halting Problem

The undecidability of the halting problem follows as an immediate corollary of Theorem 5.1. The instantiation requires ≈40 lines (defining `Terminates`, `diverge`, `halt`, and `Terminates_NonTrivial`); once those are in hand, the proof of Corollary 6.4 is one line of Rocq. The corollary adds no new mathematical content beyond the instantiation.

Definition 6.1 (Stable termination)**.** A program e *terminates stably on input* x if there exist a fuel level k and a value v such that $e(x,k) = \texttt{Some}\ v$ and $e(x,k') = \texttt{Some}\ v$ for all $k' \geq k$.

Remark 6.2 (Stable termination and the classical halting problem)**.** In the step-indexed model, stable termination and classical halting coincide: a program halts (stably) on input x if and only if some fuel level returns $\texttt{Some}\, v$. The forward direction holds because the model is monotone (`find_sol_stable`): once a value is returned, all higher fuel levels return the same value. The backward direction is immediate from the definition of stable termination. Consequently, the undecidability of `Terminates` established by Corollary 6.4 implies the undecidability of ordinary halting on input 0.

Definition 6.3 (Terminates)**.** The property `Terminates` is defined by: $\texttt{Terminates}(e) \Leftrightarrow$ e terminates stably on input 0.

`Terminates` is a semantic property in the sense of Definition 2.6: if $e \simeq f$ (observational equivalence) then $\texttt{Terminates}(e) \Leftrightarrow \texttt{Terminates}(f)$. It is non-trivial, witnessed by:

$$e_0 \;=\; \texttt{diverge} \;:=\; \lambda x.\, \lambda k.\, \texttt{None} \qquad \text{(never terminates)}$$

$$e_1 \;=\; \texttt{halt} \;:=\; \lambda x.\, \lambda k.\, \texttt{Some}\ 0 \qquad \text{(terminates stably at all fuel levels)}$$

Corollary 6.4 (Halting Problem)**.** *There is no computable function* $\mathrm{Decide}_{\mathrm{Halt}}$ *that, given a program* e*, returns* 1 *if* e *terminates stably on input* 0 *and* 0 *otherwise.*

Proof. `Terminates` is a non-trivial semantic property with witnesses $e_0 =$ `diverge` and $e_1 =$ `halt`. Apply Theorem 5.1. □ □

Remark 6.5 (Historical context and novelty)**.** Turing's 1936 proof of the halting problem's undecidability uses diagonalization: it constructs a program that halts if and only if a hypothetical decider says it does not, deriving a contradiction by self-application. That argument invokes excluded middle (the self-referential program either halts or it does not) and is therefore not valid in intuitionistic logic.

Existing constructive proofs of the halting problem in Rocq (e.g., Ciaffaglione [CG04], and the `coq-library-undecidability` [LWF22]) take the halting problem as a *seed*: they prove it undecidable first (typically by diagonalization over a Turing machine model) and then derive all other undecidability results by reduction from it.

Corollary 6.4 reverses this direction. The halting problem is derived as a *consequence* of Theorem 5.1, which is itself derived from the MRDP theorem. The undecidability of halting flows from the intractability of Diophantine arithmetic rather than from a self-referential diagonal argument. The instantiation of Rice's theorem to `Terminates` requires ≈40 lines (defining `Terminates`, `diverge`, `halt`, and `Terminates_NonTrivial`). The proof of Corollary 6.4 is then one line:

```
exact (Rice_Theorem Terminates Terminates_NonTrivial).
```

Constructive checklist. The axiom provenance of every theorem is given in Table 1 (p. 5). The sole external axiom is `H10C_SAT_undec`, proved by Larchey-Wendling and Forster [LWF22] in CIC without classical axioms; our development adds no further reasoning of any kind.

Throughout this paper the mathematical argument is fully intuitionistic. The sole external axiom, `H10C_SAT_undec`, is itself proved in CIC without classical axioms by Larchey-Wendling and Forster [LWF22], consistent with the constructive analyses of Ruitenburg [Rui85] and Cornaros–Esbelin [CE98].

(i) **LEM on $P(\bot)$.** Never determined. Lemma B uses only $P(S^1_D) \Leftrightarrow P(S^0_D)$.
(ii) **Diagonalization.** S^b_D is defined by explicit exhaustive search. No self-reference or fixed-point theorem used.
(iii) **LEM on solvability.** Structured as two separate implications, not a disjunction. δ_D makes no internal decision about solvability.
(iv) **Double-negation elimination.** Not used. The contradiction in the main proof is the direct application of the MRDP undecidability axiom to the explicitly constructed function $D \mapsto \delta_D$; no double-negation elimination is required.
(v) **Classical reasoning.** Not used. `Require Import Classical` does not appear anywhere. No `classic` or `excluded_middle` invocation appears in the development. The `LW_H10c_undec` axiom is stated in separator form (`forall cs, H10C_SAT cs -> f cs = true`) rather than the ↔ form, so no classical step is needed to convert between the two. The key supporting fact is `Lemma not_solvable_iff_unsolvable`: for `check_solution` (a `Fixpoint` returning `bool`), ¬ `is_solvable` ⇔ `is_unsolvable` holds by case analysis on the boolean, with zero axioms.
(vi) **Axiom of Choice.** Enumeration of $\mathbb{N}^k$ is canonical and explicit.

7. The Decidability Frontier for Bounded Programs

The undecidability established by Theorem 5.1 is an asymptotic result: it concerns programs with no bound on computation time or state space. In practice, programs are often analyzed under explicit resource bounds, which restores decidability. For programs bounded to terminate within T steps over state spaces of size $\leq N$, semantic properties become decidable by exhaustive enumeration — the foundation of bounded model checking.

Our reduction induces a natural *degree hierarchy* on Rice's theorem via the degree of the Diophantine witness polynomial.

Definition 7.1 (Rice degree)**.** Let $\mathsf{Rice}(d)$ denote the class of non-trivial semantic properties separable by a two-witness construction (Definition 4.2) whose Diophantine witness polynomial has degree $\leq d$. Write $\mathsf{Rice}(\infty)$ for the unrestricted class. The degree measures the minimal Diophantine complexity required to witness the undecidability of P.

Corollary 7.2 (Degree-4 saturation)**.** $\mathsf{Rice}(4) = \mathsf{Rice}(\infty)$.

Proof. Matiyasevich's MRDP encoding [Mat70] represents every r.e. set as the solution set of a degree-≤ 4 polynomial. The witness polynomial in the two-witness construction for any P can therefore be taken as the MRDP polynomial for $K = \{n \mid \phi_n(n)\downarrow\}$, which has degree ≤ 4. □

Remark 7.3 (Mechanization status)**.** Corollary 7.2 is proved mathematically above but is not yet mechanized in `rice.v`. The Rocq formalization establishes $\mathsf{Rice}(\infty)$ without the degree bound; the saturation result could be added as a Rocq corollary in a future revision.

We therefore have $\mathsf{Rice}(2) \subseteq \mathsf{Rice}(3) \subseteq \mathsf{Rice}(4) = \mathsf{Rice}(\infty)$, with the left inclusions open. Whether $\mathsf{Rice}(3) = \mathsf{Rice}(4)$ depends on whether every r.e. set has a degree-3 Diophantine representation, an open problem in Diophantine complexity [Mat93].[1]

To see that the left inclusions are not trivially collapsed by a change of witnesses, consider $\mathsf{Rice}(2)$: the class of properties witnessable by a degree-≤ 2 Diophantine polynomial. The equation $D(x, y) = x^2+y^2-n = 0$ has degree 2, and its solvability (whether n is a sum of two squares) is decidable — so no undecidability argument can be anchored there. A witness polynomial for an undecidable property must encode a Turing-complete computation, which known MRDP encodings achieve only at degree ≥ 4. Whether degree 3 suffices, and whether degree 2 can ever encode an undecidable property, are the core open questions behind the left inclusions.

Two further directions are noted briefly: the S^b_D construction is agnostic to the coefficient ring, so a positive answer to rational MRDP [Maz92, Koe16] would immediately transfer to a rational-coefficient Rice theorem with no modification to the proof; and merging the bridge lemma `LW_H10c_undec` into `coq-library-undecidability` as a PR would make Rice's theorem and the Halting Problem available as standard library lemmas.

8. Related Work

Classical proofs of Rice's theorem. The original proof by Rice [Ric53] proceeds by reduction from the halting problem, using a fixed-point argument (an application of the

[1] An unreviewed preprint [Ros25] claims a positive answer (degree-3 suffices over $\mathbb{N}$); this should be treated as open pending peer review. If confirmed, Corollary 7.2 would sharpen to $\mathsf{Rice}(3) = \mathsf{Rice}(\infty)$.

Recursion Theorem) to construct a program whose behaviour depends on the truth value of $P(\bot)$. This invokes LEM twice: once implicitly through the Recursion Theorem's diagonalization, and once through the case split $P(\bot) \vee \neg P(\bot)$. Rogers [Rog67] presents a cleaner formulation that makes the two uses of LEM explicit and is the version most commonly found in textbooks. Both proofs are essentially classical and do not transfer directly to intuitionistic logic.

Constructive computability theory and realizability. The general programme of constructivising classical computability results is surveyed by Beeson [Bee85] and treated systematically by Troelstra and van Dalen [TvD88]. Within intuitionistic mathematics, undecidability statements split into two strengths: $\neg(\exists e,\, \mathrm{Decide}_P = e)$ (our form: no decider exists) versus the stronger $\forall e,\, \neg(\mathrm{Decide}_P = e)$ (every program fails to decide P). Our proof establishes the former.

From the realizability perspective [Bau06], our result holds in *any* partial combinatory algebra: the two-witness construction is entirely explicit (no oracle, no fixed-point combinator), so it realises the undecidability statement directly. In the effective topos [Hyl82], where all functions are computable, Rice's theorem becomes an internal theorem with no metatheoretic overhead.

The *synthetic* approach to computability, developed by Forster et al. [FKS19] in Coq and extended by Ciaffaglione et al. [CG04], axiomatizes computability internally to type theory. Our proof is compatible with this approach: replacing our step-indexed programs with any synthetic notion of computability satisfying observational extensionality would yield the same result, since we use no property of the model beyond Definition 2.1.

Our proof falls in the weakest constructive category: valid in intuitionistic predicate logic with the metatheoretic assumptions on MRDP stated in Section 3, and no further classical principles.

The MRDP theorem and constructive arithmetic. The Davis–Putnam–Robinson–Matiyasevich theorem [DPR61, Mat70] is classical in its original formulation. Its constructive status was first analysed by Ruitenburg [Rui85], who showed it holds in intuitionistic arithmetic with the axiom of choice for numbers. Cornaros and Esbelin [CE98] provide a detailed treatment of the constructive content of Hilbert's Tenth Problem, establishing the precise fragment of intuitionistic arithmetic required. A full mechanization of MRDP in Rocq has been given by Larchey-Wendling and Forster [LWF22], confirming its constructive validity in Coq's type theory. Our proof takes `MRDP_undecidable` as an axiom that acts as a pointer to that mechanization; it does not depend on the details of the constructivization beyond the existence of the many-one reduction from K to Diophantine solvability. Crucially, `LW_H10c_undec` is a *negative proposition* ($\neg\, \exists f, \ldots$); its proof in CIC proceeds by intuitionistically valid reductio (deriving $\bot$ from the assumed decider) without invoking LEM on any Diophantine instance. This is the precise sense in which the axiom is compatible with the purely intuitionistic metatheory of the rest of the development.

Step-indexed models and observational equivalence. Step-indexed models of computation were introduced by Appel and McAllester [AM01] in the context of type-theoretic reasoning about recursive types, and have since become a standard tool for mechanized semantics. Our use of the step-indexed model is more elementary: we require only that programs are functions $\mathbb{N} \to (\mathbb{N} \to \mathrm{option}\,\mathbb{N})$ and that convergence is monotone in fuel. The

observational equivalence of Definition 2.1 is a fuel-indexed variant of the standard notion used in Pitts [Pit02] and related work on operational semantics.

Mechanized proofs of undecidability and comparison with Forster et al. The `coq-library-undecidability` [FKS19, LWF22], maintained on GitHub,[2] contains a full Rocq mechanization of the MRDP theorem by Larchey-Wendling and Forster [LWF22]. That development uses Minsky machines as the computation model and Conway's FRACTRAN language as an intermediate layer. It is built on the synthetic computability framework of Forster [For22], in which `undecidable` P denotes `decidable` $P \to$ `enumerable`$(\overline{\texttt{SBTM_HALT}})$ rather than $\neg$ `decidable` P; this is the reason the bridge proof requires no classical axioms. Our `MRDP_undecidable` hypothesis is precisely the statement they prove; our axiom is therefore a pointer to their library rather than a genuine assumption. Connecting the two formalizations directly would require bridging our step-indexed computation model with their Minsky machine model, which we leave as future work.

Forster, Kirst, and Smolka [FKS19] also formalize Rice's theorem in Coq as part of that library. Their proof follows the classical Rogers presentation [Rog67]: given a non-trivial property P with witness f_1 satisfying P, they construct for each program e a new program h_e defined by: simulate e on some canonical input; if e halts, behave like f_1; otherwise behave like f_0 (or the reverse, depending on whether $P(\bot)$ holds). This construction requires two classical steps. *First*, h_e must "know" its own index in order to refer to e; this is implemented via the s-1-1 theorem, which relies on the Recursion Theorem and therefore on a diagonalization argument. *Second*, the direction of the reduction depends on whether $P(\bot)$ holds, so their proof performs an explicit case split on $P(\bot) \vee \neg P(\bot)$, invoking LEM. Their formalization uses the `Classical` library throughout.

Our proof eliminates both steps. The witness programs S_D^b are constructed from a Diophantine polynomial D and a bit b alone — no self-reference, no reference to e, no fixed-point argument. The $P(\bot)$ case split is avoided by design: when D is unsolvable, S_D^0 and S_D^1 are *observationally identical* (both diverge everywhere), so $P(S_D^0) \Leftrightarrow P(S_D^1)$ follows by extensionality with no knowledge of $P(\bot)$. The axiom audit is given in Table 1. Thus, while their proof is classical, ours remains intuitionistic throughout.

Bounded decidability and model checking. The connection between bounded program analysis and decidability has been studied extensively in the model checking literature. Clarke et al. [CHVB18] survey bounded model checking and its completeness thresholds. The relationship between the complexity of semantic properties and the parameters (T, N) of the bounded model (termination steps and state space size) is closely related to questions in descriptive complexity studied by Immerman [Imm99]. The open problem stated in Section 7 — characterizing the decidability frontier as a function of (T, N) — appears to be new. The precise position of Rice's theorem in the Weihrauch lattice [BGP21] is a further open question; $\mathsf{C}_{\mathbb{N}}$ (closed choice on $\mathbb{N}$) is a natural candidate but we do not establish either direction of the equivalence here.

9. Conclusion

We have presented a constructive proof of Rice's theorem via MRDP that:

(1) uses Hilbert's Tenth Problem as its undecidability anchor;

[2] `https://github.com/uds-psl/coq-library-undecidability`

(2) avoids LEM, diagonalization, and any assumption about $P(\bot)$;
(3) relies on a two-witness construction where $\delta_D = a_D - b_D$ encodes solvability without ever evaluating $P(\bot)$;
(4) is structured as two implications each valid in intuitionistic logic;
(5) imposes no totality condition on the witnesses, covering the full classical statement of Rice's theorem for arbitrary partial programs.

A Rocq formalization is provided in the appendix confirming the constructive validity of the argument in a purely intuitionistic setting, with the undecidability of Hilbert's Tenth Problem as the only axiom.

The constructiveness of the proof has a practical consequence. The S_D^b witness programs are not abstract objects — they can be written down. The file `undecidable.c`, available at `https://github.com/endrazine/rice-constructive`, is a single C program with two modes. The first mode searches for solutions to $x^2 - y^2 - z^2 = 1$ (solvable; terminates provably). The second mode searches sequentially, for $k = 0, 1, 2, \ldots$ up to $2^{64} - 1$ (stored as `uint64_t`, platform-independent; see Remark 9.1 for the rationale) (skipping $k \equiv 4$ or $5 \pmod 9$, which are provably impossible), for integer solutions to $x^3+y^3+z^3 = k$, advancing to $k + 1$ as soon as a solution for the current k is found. This program terminates if and only if every integer not congruent to 4 or 5 modulo 9 is a sum of three integer cubes — an open conjecture in number theory as of 2026. Unlike a search for a single fixed target, this formulation is permanently open: resolving any individual k merely advances the search, and the termination question is equivalent to the full conjecture. In both cases, Theorem 5.1 implies that no static analyzer can correctly decide termination across the full family of such programs: any analyzer that could would decide arbitrary Diophantine solvability, which is impossible by MRDP.

Remark 9.1 (Correspondence to S_D^b)**.** The structure of `undecidable.c` mirrors S_D^b directly. Both programs share a generic search engine `find_sol_one()` that enumerates tuples $\vec{x} \in \mathbb{N}^k$ via Cantor decoding (steps 1–2 of Definition 4.2), evaluates the polynomial check `D(x)==0`, and on success calls the target behavior (step 3). The `decidable` program is a ground instance with $b = 1$, $e_1 =$ `halt`, and the polynomial $D(x, y, z) = x^2 - y^2 - z^2 - 1$ from Example 4.1. The `undecidable` program composes `find_sol_one()` instances sequentially over $k = 0, 1, 2, \ldots$, yielding a program whose termination is equivalent to a conjecture in number theory rather than to any single Diophantine instance; resolving any individual k merely advances the search. The outer loop counter `k` is a `uint64_t` (platform-independent, max $2^{64}-1$); this is intentional — the program stalls at the first open value of the conjecture and never approaches $2^{64}-1$ in practice, so overflow is purely hypothetical. The inner tuple counter `n` inside `find_sol_one()` uses GMP `mpz_t` as a correctness requirement: a fixed-width counter would exhaust in finite time and invalidate the formal claim. `undecidable.c` is a hand-written illustration, not a formally extracted program. Formal extraction from the Rocq development (e.g., via `Extraction` to OCaml, then compilation) is feasible and is left for a companion artefact.

Formalization design decisions. Five choices were critical to making the intuitionistic proof work in Rocq: (1) full fuel pass-through in `S_D`, eliminating the totality condition on witnesses; (2) a $\mathbb{Z}$-valued decider, avoiding `nat` subtraction truncation; (3) `not_solvable_iff_unsolvable`, proved by `destruct` on the boolean with zero axioms; (4) splitting $\mathbb{Z}$-coefficient polynomials into positive and negative `nat` parts for the arithmetic encoding; and (5) the bridge via

`h10c_to_Poly` composing separators rather than inverting the encoding. Each is discussed at its point of use in Appendix A.

Acknowledgements

The author thanks Claude (Anthropic) for assistance with Rocq proof drafting and structural feedback on earlier versions of this manuscript.

10. Formalization in Rocq

The following excerpt shows the key axiom, theorems, and their Rocq statements. The complete self-contained development (1919 lines, zero admitted goals, zero classical axioms) is available at:

`https://github.com/endrazine/rice-constructive`

The formalization uses a step-indexed model of computation. Diophantine polynomials of arbitrary arity are handled via a k-ary Cantor pairing function encoded as `decode_k`; the arity is threaded as the parameter `ar`. The decider output is represented using `Z` (see §5). The key constructive lemma is `not_solvable_iff_unsolvable`: for the computable `bool`-returning `check_solution`, the equivalence $\neg$ `is_solvable` $\Leftrightarrow$ `is_unsolvable` holds with zero axioms by case analysis on the boolean. The sole external axiom is `H10C_SAT_undec`; `LW_H10c_undec` is proved from it with zero admitted goals. The axiom provenance of every theorem is summarised in Table 1.

The key design choice eliminating all classical reasoning is that the witness programs S_D^b pass their full fuel budget to e_b unchanged (Definition of `S_D`). When the Diophantine solution is found at step m, for all fuel $\geq m$ we have $S_D^b(x, \text{fuel}) = e_b(x, \text{fuel})$ exactly, so observational equivalence holds with $N = m$ and no fuel arithmetic is needed. This eliminates the fuel-offset problem that previously required a totality condition on the witnesses, making the theorem valid for arbitrary partial programs e_0 and e_1.

Table 2: Correspondence between Paper Concepts and Rocq Definitions

Paper Concept	**Rocq Definition**
Program	`Program := nat -> nat -> option nat`
Convergence	`p x fuel = Some v`
Observational equivalence	`obs_equiv p1 p2`
Semantic property	`SemanticProperty`
Non-triviality	`NonTrivial sp`
Witness programs (arity `ar`, poly `D`)	`S_D sp wit ar D b`
MRDP undecidability	`MRDP_undecidable` (axiom)

Appendix A. Complete Rocq Formalization

The following is the complete, self-contained Rocq formalization. It compiles with zero errors and zero admitted goals under Rocq (Coq) 8.18 or later, producing:

```
Print Assumptions LW_H10c_undec.   (* H10C_SAT_undec *)
Print Assumptions Rice_Theorem.    (* Closed under the global context *)
Print Assumptions Halting_Problem. (* Closed under the global context *)
```

```
(** * A Constructive Proof of Rice's Theorem via Hilbert's Tenth Problem
    Author: Jonathan Brossard <brossardj@acm.org>

    This file contains the complete Rocq (Coq) formalization accompanying
    the paper of the same title.

    AXIOMS:
    - H10C_SAT_undec: undecidability of H10C (from coq-library-undecidability,
      proved via FRACTRAN -> Minsky Machines -> Halting -> MRDP chain)

    THEOREM HIERARCHY:
    - H10C_SAT_undec   (axiom: from coq-library-undecidability)
        --> bridge: H10C_SAT <= H10c_sat
    - LW_H10c_undec    (proved: 0 admits)
        --> poly_encode_correct (proved: 0 axioms)
    - MRDP_from_LW     (proved: 0 axioms once LW proved)
        --> S_D construction
    - Rice_Theorem     (proved: closed under global context)
        --> instantiation
    - Halting_Problem  (proved: closed under global context)

    Tested with Rocq (Coq) 8.18 or later.
*)

Require Import Arith List Bool Lia ZArith Ring.
Import ListNotations.


Section Rice_Theorem.

  (* ================================================================ *)
  (* 1. DIOPHANTINE INFRASTRUCTURE (k-ary Cantor encoding)            *)
  (* ================================================================ *)

  Definition Monomial := (Z * list nat)%type.
  Definition Poly := list Monomial.

  Fixpoint pow (base exp : nat) : nat :=
    match exp with
    | O   => 1%nat
    | S e => (base * pow base e)%nat
    end.

  Definition eval_monomial (m : Monomial) (vars : list nat) : Z :=
    let (coeff, exps) := m in
    let term := fold_left
                  (fun acc p => (acc * Z.of_nat (pow (fst p) (snd p)))%Z)
                  (combine vars exps) 1%Z in
    (coeff * term)%Z.

  Definition eval_poly (p : Poly) (vars : list nat) : Z :=
    fold_left (fun acc m => (acc + eval_monomial m vars)%Z) p 0%Z.

  Definition cantor_w (n : nat) : nat :=
    (Nat.sqrt (8 * n + 1) - 1) / 2.

  Definition cantor_pair (a b : nat) := (a+b)*(a+b+1)/2 + b.
```

```
Definition cantor_unpair_fst (n : nat) : nat :=
  let w := cantor_w n in
  w - (n - w * (w + 1) / 2).

Definition cantor_unpair_snd (n : nat) : nat :=
  let w := cantor_w n in
  n - w * (w + 1) / 2.

Fixpoint decode_k (ar : nat) (n : nat) : list nat :=
  match ar with
  | O    => @nil nat
  | S O  => n :: nil
  | S a' => decode_k a' (cantor_unpair_fst n) ++ (cantor_unpair_snd n :: nil)
  end.

Definition check_solution (ar : nat) (p : Poly) (n : nat) : bool :=
  if Z.eq_dec (eval_poly p (decode_k ar n)) 0%Z then true else false.

Definition is_solvable (ar : nat) (p : Poly) :=
  exists n, check_solution ar p n = true.
Definition is_unsolvable (ar : nat) (p : Poly) :=
  forall n, check_solution ar p n = false.

(* ================================================================ *)
(* 2. COMPUTATION MODEL                                             *)
(* ================================================================ *)

Definition Program := nat -> nat -> option nat.

Definition Stable (p : Program) : Prop :=
  forall x k v, p x k = Some v ->
    forall k', k' >= k -> p x k' = Some v.

Definition obs_equiv (p1 p2 : Program) : Prop :=
  forall x, exists N, forall k, k >= N -> p1 x k = p2 x k.

Notation "p1 ~= p2" := (obs_equiv p1 p2) (at level 70).

Lemma obs_equiv_refl : forall p, p ~= p.
Proof.
  intros p x. exists 0%nat%nat. intros k _. reflexivity.
Qed.

Lemma obs_equiv_sym : forall p1 p2, p1 ~= p2 -> p2 ~= p1.
Proof.
  intros p1 p2 H x.
  destruct (H x) as [N HN].
  exists N. intros k Hk. symmetry. apply HN. exact Hk.
Qed.

Lemma obs_equiv_trans : forall p1 p2 p3,
    p1 ~= p2 -> p2 ~= p3 -> p1 ~= p3.
Proof.
  intros p1 p2 p3 H12 H23 x.
  destruct (H12 x) as [N1 HN1].
  destruct (H23 x) as [N2 HN2].
  exists (Nat.max N1 N2). intros k Hk.
  transitivity (p2 x k);
    [apply HN1 | apply HN2]; lia.
Qed.

(* ================================================================ *)
(* 3. SEARCH DYNAMICS                                               *)
(* ================================================================ *)

Fixpoint find_sol (ar : nat) (p : Poly) (fuel : nat) : option nat :=
```

```
  match fuel with
  | 0       => if check_solution ar p 0 then Some 0 else None
  | S fuel' => match find_sol ar p fuel' with
               | Some n => Some n
               | None   =>
                 if check_solution ar p (S fuel')
                 then Some (S fuel') else None
               end
  end.

Lemma find_sol_step : forall ar p j n,
  find_sol ar p j = Some n -> find_sol ar p (S j) = Some n.
Proof.
  intros ar p j n Hj. simpl. rewrite Hj. reflexivity.
Qed.

Lemma find_sol_stable : forall ar p fuel n,
  find_sol ar p fuel = Some n ->
  forall fuel', fuel' >= fuel -> find_sol ar p fuel' = Some n.
Proof.
  intros ar p fuel n H fuel' Hle.
  induction fuel' as [|f'' IHf''].
  - assert (fuel = 0) by lia. subst. exact H.
  - destruct (Nat.eq_dec fuel (S f'')) as [-> | Hne].
    + exact H.
    + apply find_sol_step. apply IHf''. lia.
Qed.

(* Helper: if find_sol ar p n = Some m then m <= n *)
Lemma find_sol_bound : forall ar p n m,
  find_sol ar p n = Some m -> m <= n.
Proof.
  intros ar p n.
  induction n; intros m H.
  - simpl in H.
    destruct (check_solution ar p 0%nat) eqn:C.
    + injection H as <-. lia.
    + discriminate.
  - simpl in H.
    revert H.
    case_eq (find_sol ar p n).
    + intros n0 En0 H.
      injection H as <-.
      apply Nat.le_le_succ_r. apply IHn. exact En0.
    + intros En.
      destruct (check_solution ar p (S n)) eqn:C.
      * intros H. injection H as <-. lia.
      * intros H. discriminate.
Qed.

Lemma find_sol_at_n : forall ar p n,
  check_solution ar p n = true ->
  exists m, m <= n /\ find_sol ar p n = Some m.
Proof.
  induction n; intros H.
  - exists 0%nat. split; [lia | simpl; rewrite H; auto].
  - simpl. destruct (find_sol ar p n) eqn:E.
    + exists n0. split; [| reflexivity].
      apply Nat.le_le_succ_r. apply find_sol_bound with (ar := ar) (p := p). exact E.
    + exists (S n). split; [lia | rewrite H; reflexivity].
Qed.

Lemma find_sol_unsolvable : forall ar p,
  is_unsolvable ar p -> forall fuel, find_sol ar p fuel = None.
Proof.
  intros ar p Hunsol fuel. induction fuel.
  - simpl. rewrite Hunsol. reflexivity.
```

```
  - simpl. rewrite IHfuel. rewrite Hunsol. reflexivity.
Qed.

(* ================================================================ *)
(* 4. SEMANTIC PROPERTIES                                           *)
(* ================================================================ *)

Record SemanticProperty := {
  P_pred         : Program -> Prop;
  extensionality : forall e f, e ~= f -> (P_pred e <-> P_pred f)
}.

(** No Stable condition on e0/e1: S_D passes full fuel to e_b
    unchanged, so no monotonicity of witnesses is required. *)
Record NonTrivial (sp : SemanticProperty) := {
  e0       : Program;
  e1       : Program;
  not_P_e0 : ~ P_pred sp e0;
  P_e1     : P_pred sp e1
}.

(* ================================================================ *)
(* 5. TWO-WITNESS CONSTRUCTION                                      *)
(* ================================================================ *)

(** Key design: pass the full fuel budget to e_b unchanged.
    This avoids any fuel-offset reasoning and requires no
    totality condition on the witnesses. *)
Definition S_D (sp : SemanticProperty) (wit : NonTrivial sp)
               (ar : nat) (D : Poly) (b : bool) : Program :=
  fun x fuel =>
    match find_sol ar D fuel with
    | Some _ => if b then e1 sp wit x fuel
                     else e0 sp wit x fuel
    | None   => None
    end.

(** When D is solvable, find_sol stabilizes at step m, so for
    all fuel >= m we have S_D x fuel = e_b x fuel exactly. *)
Lemma S_D_solvable_equiv : forall sp wit ar D b n,
  check_solution ar D n = true ->
  S_D sp wit ar D b ~= (if b then e1 sp wit else e0 sp wit).
Proof.
  intros sp wit ar D b n Hn.
  destruct (find_sol_at_n ar D n Hn) as [m [_ Hfind]].
  (* Hfind : find_sol ar D n = Some m
     For fuel >= n, find_sol ar D fuel = Some m by find_sol_stable,
     so S_D x fuel = e_b x fuel. *)
  destruct b.
  - intros x. exists n. intros fuel Hfuel. unfold S_D.
    rewrite (find_sol_stable ar D n m Hfind fuel Hfuel).
    reflexivity.
  - intros x. exists n. intros fuel Hfuel. unfold S_D.
    rewrite (find_sol_stable ar D n m Hfind fuel Hfuel).
    reflexivity.
Qed.

Lemma S_D_unsolvable_equiv : forall sp wit ar D,
  is_unsolvable ar D ->
  S_D sp wit ar D true ~= S_D sp wit ar D false.
Proof.
  intros sp wit ar D Hunsol x.
  exists 0%nat%nat. intros fuel _. unfold S_D.
  rewrite (find_sol_unsolvable ar D Hunsol fuel). reflexivity.
Qed.

(* ================================================================ *)
```

```
(* 6. BRIDGE TO LARCHEY-WENDLING                                    *)
(*                                                                  *)
(* This section mechanizes the logical connection between the       *)
(* MRDP_undecidable axiom below and the constructive undecidability *)
(* of Hilbert's Tenth Problem proved by Larchey-Wendling & Forster: *)
(*                                                                  *)
(*   "Hilbert's Tenth Problem in Coq (Extended Version)"            *)
(*   LMCS 18(1):35, 2022.  DOI: 10.46298/lmcs-18(1:35)2022         *)
(*   https://github.com/uds-psl/coq-library-undecidability          *)
(*                                                                  *)
(* The four lemmas in Section 6.1 require NO axioms and are fully   *)
(* constructive.  Section 6.2 proves poly_encode_correct (zero axioms). *)
(* (an arithmetic encoding of our Poly type into their h10c format  *)
(* and the import of their theorem) as explicit Axioms, and proves  *)
(* MRDP_undecidable from them.  Once those Axioms are replaced by   *)
(* proved lemmas, MRDP_undecidable below becomes axiom-free.        *)
(* ================================================================ *)

(* ---------------------------------------------------------------- *)
(* 6.1 FUNDAMENTAL DUALITY AND LOGICAL BRIDGE (no axioms)           *)
(* ---------------------------------------------------------------- *)

(** is_unsolvable is the constructive negation of is_solvable.
    Both unfold to the same predicate over the bool function
    check_solution.  The equivalence uses only:
      ~(exists n, b n = true) <-> forall n, b n = false
    which holds in intuitionistic logic for any bool-valued function. *)
(** For bool check_solution: ~is_solvable <-> is_unsolvable. Purely constructive. *)
Lemma not_solvable_iff_unsolvable : forall ar p,
  ~ is_solvable ar p <-> is_unsolvable ar p.
Proof.
  intros ar p. unfold is_solvable, is_unsolvable.
  split.
  - (* ~(exists n, check n = true) -> forall n, check n = false *)
    intros Hno n.
    destruct (check_solution ar p n) eqn:E.
    + exfalso. apply Hno. exists n. exact E.
    + reflexivity.
  - intros Hall [n Hn].
    rewrite (Hall n) in Hn. discriminate.
Qed.

Lemma unsolvable_iff_not_solvable : forall ar p,
  is_unsolvable ar p <-> ~ is_solvable ar p.
Proof.
  intros ar p. unfold is_solvable, is_unsolvable. split.
  - intros Huns [n Hn]. rewrite Huns in Hn. discriminate.
  - intros Hnot n.
    destruct (check_solution ar p n) eqn:E.
    + exfalso. apply Hnot. exists n. exact E.
    + reflexivity.
Qed.

(** A Z-valued (1/0) separator for our Poly type. *)
Definition Z_separator (F : nat -> Poly -> Z) : Prop :=
  (forall ar p, is_solvable   ar p -> F ar p = 1%Z) /\
  (forall ar p, is_unsolvable ar p -> F ar p = 0%Z).

(** A boolean separator for our Poly type. *)
Definition bool_separator (f : nat -> Poly -> bool) : Prop :=
  (forall ar p, is_solvable   ar p -> f ar p = true)  /\
  (forall ar p, is_unsolvable ar p -> f ar p = false).

(** KEY LEMMA: Every Z-separator yields a boolean separator.
    Given F, define f ar p = Z.eqb (F ar p) 1.
    The separation conditions hold by rewriting alone.
    No double-negation elimination. No excluded middle. *)
```

```
Lemma Z_sep_to_bool_sep :
  forall (F : nat -> Poly -> Z),
    Z_separator F -> bool_separator (fun ar p => Z.eqb (F ar p) 1).
Proof.
  intros F [Hsol Hunsol]. split.
  - intros ar p H. rewrite (Hsol ar p H). reflexivity.
  - intros ar p H. rewrite (Hunsol ar p H). reflexivity.
Qed.

(** BRIDGE THEOREM: Absence of a boolean separator implies absence
    of a Z-separator.  Contrapositive of Z_sep_to_bool_sep.
    No classical reasoning. *)
Theorem bridge_logical :
  (~ exists f, bool_separator f) ->
  (~ exists F, Z_separator    F).
Proof.
  intros Hnobool [F HF].
  apply Hnobool.
  exists (fun ar p => Z.eqb (F ar p) 1).
  apply Z_sep_to_bool_sep. exact HF.
Qed.

(** Unfolded form matching the MRDP_undecidable statement below. *)
Theorem bridge_logical_unfolded :
  (~ exists f : nat -> Poly -> bool,
       (forall ar p, is_solvable   ar p -> f ar p = true)  /\
       (forall ar p, is_unsolvable ar p -> f ar p = false)) ->
  (~ exists F : nat -> Poly -> Z,
       (forall ar p, is_solvable   ar p -> F ar p = 1%Z) /\
       (forall ar p, is_unsolvable ar p -> F ar p = 0%Z)).
Proof.
  intros Hnobool [F [Hsol Hunsol]].
  apply Hnobool.
  exists (fun ar p => Z.eqb (F ar p) 1). split.
  - intros ar p H. rewrite (Hsol ar p H). reflexivity.
  - intros ar p H. rewrite (Hunsol ar p H). reflexivity.
Qed.

(* ---------------------------------------------------------------- *)
(* 6.2 ENCODING OBLIGATION AND LW CONNECTION                        *)
(*                                                                  *)
(* Larchey-Wendling's library works with h10c constraints:          *)
(*   x = 1  |  x = y + z  |  x = y * z                            *)
(* To connect our Poly type to their theorem, we need:              *)
(*   (a) An encoding: poly_encode : nat -> Poly -> h10c_inst        *)
(*       such that is_solvable ar p <-> H10c_sat (poly_encode ar p) *)
(*   (b) An import of their undecidability theorem.                 *)
(*                                                                  *)
(* poly_encode_correct is fully proved (zero axioms, zero admits).  *)
(* The encoding is a routine arithmetic exercise connecting our Poly  *)
(* type to the h10c constraint format.                               *)
(* Hypothesis MRDP_undecidable is discharged by Theorem MRDP_from_LW *)
(* and carries zero classical axioms.                                *)
(* ---------------------------------------------------------------- *)

(** The following proves poly_encode_correct with no axioms.
    h10c_inst, H10c_sat, poly_encode, and poly_encode_correct are all
    concrete definitions and theorems. LW_H10c_undec remains as the
    single remaining axiom (one Import from coq-library-undecidability). *)

(* ================================================================ *)
(* 2. NAT-COEFFICIENT POLYNOMIALS                                   *)
(* ================================================================ *)

Definition NatMonomial := (nat * list nat)%type.
Definition NatPoly     := list NatMonomial.
```

```
Fixpoint eval_nat_mono_aux (exps vals : list nat) (acc : nat) : nat :=
  match exps, vals with
  | [], _ | _, [] => acc
  | e::es, v::vs  => eval_nat_mono_aux es vs (acc * pow v e)
  end.

Definition eval_nat_mono (m : NatMonomial) (vars : list nat) : nat :=
  fst m * eval_nat_mono_aux (snd m) vars 1.

Definition eval_nat_poly (p : NatPoly) (vars : list nat) : nat :=
  fold_left (fun acc m => (acc + eval_nat_mono m vars)%nat) p 0%nat.

(* ================================================================ *)
(* 3. FILTER-OPT                                                    *)
(* ================================================================ *)

Fixpoint filter_opt {A B : Type} (f : A -> option B) (l : list A) : list B :=
  match l with
  | []     => []
  | h :: t => match f h with
              | None   => filter_opt f t
              | Some x => x :: filter_opt f t
              end
  end.

Definition pos_monomials (p : Poly) : NatPoly :=
  filter_opt (fun m =>
    if Z.ltb 0%Z (fst m) then Some (Z.to_nat (fst m), snd m) else None) p.

Definition neg_monomials (p : Poly) : NatPoly :=
  filter_opt (fun m =>
    if Z.ltb (fst m) 0%Z then Some (Z.to_nat (Z.opp (fst m)), snd m) else None) p.

(* ================================================================ *)
(* 4. pos/neg_monomials STEP LEMMAS                                 *)
(* ================================================================ *)

Lemma pos_step_pos : forall h t, (0 < fst h)%Z ->
  pos_monomials (h :: t) = (Z.to_nat (fst h), snd h) :: pos_monomials t.
Proof.
  intros h t H. unfold pos_monomials; simpl.
  replace (Z.ltb 0%Z (fst h)) with true by (symmetry; apply Z.ltb_lt; lia).
  reflexivity. Qed.

Lemma neg_step_pos : forall h t, (0 < fst h)%Z ->
  neg_monomials (h :: t) = neg_monomials t.
Proof.
  intros h t H. unfold neg_monomials; simpl.
  replace (Z.ltb (fst h) 0%Z) with false by (symmetry; apply Z.ltb_nlt; lia).
  reflexivity. Qed.

Lemma pos_step_neg : forall h t, (fst h < 0)%Z ->
  pos_monomials (h :: t) = pos_monomials t.
Proof.
  intros h t H. unfold pos_monomials; simpl.
  replace (Z.ltb 0%Z (fst h)) with false by (symmetry; apply Z.ltb_nlt; lia).
  reflexivity. Qed.

Lemma neg_step_neg : forall h t, (fst h < 0)%Z ->
  neg_monomials (h :: t) = (Z.to_nat (Z.opp (fst h)), snd h) :: neg_monomials t.
Proof.
  intros h t H. unfold neg_monomials; simpl.
  replace (Z.ltb (fst h) 0%Z) with true by (symmetry; apply Z.ltb_lt; lia).
  reflexivity. Qed.

Lemma pos_step_zero : forall h t, (fst h = 0)%Z ->
  pos_monomials (h :: t) = pos_monomials t.
```

```
Proof.
  intros h t H. unfold pos_monomials; simpl.
  replace (Z.ltb 0%Z (fst h)) with false by (symmetry; apply Z.ltb_nlt; lia).
  reflexivity. Qed.

Lemma neg_step_zero : forall h t, (fst h = 0)%Z ->
  neg_monomials (h :: t) = neg_monomials t.
Proof.
  intros h t H. unfold neg_monomials; simpl.
  replace (Z.ltb (fst h) 0%Z) with false by (symmetry; apply Z.ltb_nlt; lia).
  reflexivity. Qed.

(* ================================================================ *)
(* 5. FOLD ACCUMULATOR LEMMAS                                       *)
(* ================================================================ *)

Lemma fold_Zadd_natmono_acc : forall (vars : list nat) (l : list NatMonomial) (init : Z),
  fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) l init =
  (init + fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) l 0%Z)%Z.
Proof.
  intros vars l. induction l as [| h t IH]; intro init; simpl.
  - lia.
  - rewrite (IH (init + Z.of_nat (eval_nat_mono h vars))%Z).
    rewrite (IH (Z.of_nat (eval_nat_mono h vars))). lia.
Qed.

(** fold_natmono_step: unfold one step of the NatMonomial fold. *)
Lemma fold_natmono_step : forall (vars : list nat) (n : nat) (exps : list nat)
                                 (l : list NatMonomial) (init : Z),
  fold_left (fun a m => (a + Z.of_nat (eval_nat_mono m vars))%Z)
            ((n, exps) :: l) init =
  (init + Z.of_nat (eval_nat_mono (n, exps) vars) +
   fold_left (fun a m => (a + Z.of_nat (eval_nat_mono m vars))%Z) l 0%Z)%Z.
Proof.
  intros vars n exps l init. simpl.
  rewrite (fold_Zadd_natmono_acc vars l _). lia.
Qed.

Lemma fold_Zmul_acc : forall (l : list (nat * nat)) (f : nat * nat -> nat) (init : Z),
  fold_left (fun acc p => (acc * Z.of_nat (f p))%Z) l init =
  (init * fold_left (fun acc p => (acc * Z.of_nat (f p))%Z) l 1%Z)%Z.
Proof.
  induction l as [| h t IH]; intros f init.
  - simpl. ring.
  - simpl. rewrite (IH f (init * Z.of_nat (f h))%Z).
    destruct (Z.of_nat (f h)) as [| p | p].
    + simpl. rewrite (IH f 0%Z). ring.
    + rewrite (IH f (Z.pos p)). ring.
    + rewrite (IH f (Z.neg p)). ring.
Qed.

Lemma fold_natmul_to_Z : forall (l : list (nat * nat)) (f : nat * nat -> nat) (init : nat),
  Z.of_nat (fold_left (fun acc p => (acc * f p)%nat) l init) =
  (Z.of_nat init * fold_left (fun acc p => (acc * Z.of_nat (f p))%Z) l 1%Z)%Z.
Proof.
  induction l as [| h t IH]; intros f init; simpl.
  - ring.
  - rewrite IH. rewrite Nat2Z.inj_mul.
    pose proof (Nat2Z.is_nonneg (f h)) as Hnn.
    destruct (Z.of_nat (f h)) as [| p | p].
    + simpl. rewrite (fold_Zmul_acc t f 0%Z). ring.
    + rewrite (fold_Zmul_acc t f (Z.pos p)). ring.
    + lia.
Qed.

(* ================================================================ *)
(* 6. MONOMIAL VALUE LEMMAS                                         *)
```

```
(* ================================================================ *)

(* Compatibility: eval_nat_mono_aux equals the fold_left/combine form *)
Lemma eval_nat_mono_aux_eq : forall exps vars acc,
  eval_nat_mono_aux exps vars acc =
  fold_left (fun a p => (a * pow (fst p) (snd p))%nat) (combine vars exps) acc.
Proof.
  induction exps as [|e es IH]; intros vars acc.
  - destruct vars; simpl; reflexivity.
  - destruct vars as [|v vs]. { simpl. reflexivity. }
    simpl. rewrite IH. reflexivity.
Qed.

Lemma pos_mono_val : forall c exps vars,
  (0 < c)%Z ->
  Z.of_nat (eval_nat_mono (Z.to_nat c, exps) vars) = eval_monomial (c, exps) vars.
Proof.
  intros c exps vars Hpos.
  unfold eval_nat_mono, eval_monomial; simpl.
  rewrite Nat2Z.inj_mul, Z2Nat.id by lia.
  f_equal. rewrite eval_nat_mono_aux_eq.
  rewrite (fold_natmul_to_Z (combine vars exps) _ 1%nat). ring.
Qed.

Lemma neg_mono_val : forall c exps vars,
  (c < 0)%Z ->
  Z.of_nat (eval_nat_mono (Z.to_nat (Z.opp c), exps) vars) =
  (- eval_monomial (c, exps) vars)%Z.
Proof.
  intros c exps vars Hneg.
  unfold eval_nat_mono, eval_monomial; simpl.
  rewrite Nat2Z.inj_mul, Z2Nat.id by lia.
  rewrite eval_nat_mono_aux_eq.
  rewrite (fold_natmul_to_Z (combine vars exps) _ 1%nat). ring.
Qed.

(* ================================================================ *)
(* 7. eval_nat_poly LIFT                                            *)
(* ================================================================ *)

Lemma fold_natadd_to_Z : forall (l : list NatMonomial) (vars : list nat) (init : nat),
  Z.of_nat (fold_left (fun acc m => (acc + eval_nat_mono m vars)%nat) l init) =
  (Z.of_nat init +
   fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) l 0%Z)%Z.
Proof.
  induction l as [| h t IH]; intros vars init; simpl.
  - lia.
  - rewrite (IH vars (init + eval_nat_mono h vars)%nat).
    rewrite Nat2Z.inj_add.
    rewrite (fold_Zadd_natmono_acc vars t (Z.of_nat (eval_nat_mono h vars))). lia.
Qed.

Lemma eval_nat_poly_to_Z : forall p vars,
  Z.of_nat (eval_nat_poly p vars) =
  fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) p 0%Z.
Proof.
  intros p vars. unfold eval_nat_poly.
  rewrite (fold_natadd_to_Z p vars 0%nat). lia.
Qed.

(* ================================================================ *)
(* 8. THE MAIN SPLIT THEOREM                                        *)
(* ================================================================ *)

Lemma eval_poly_split : forall (p : Poly) vars,
  eval_poly p vars =
  (Z.of_nat (eval_nat_poly (pos_monomials p) vars) -
```

```
    Z.of_nat (eval_nat_poly (neg_monomials p) vars))%Z.
Proof.
  intros p vars.
  rewrite !eval_nat_poly_to_Z. unfold eval_poly.
  assert (KEY : forall l (az : Z),
    fold_left (fun a m => (a + eval_monomial m vars)%Z) l az =
    (az + fold_left (fun a m => (a + Z.of_nat (eval_nat_mono m vars))%Z)
                    (pos_monomials l) 0%Z
       - fold_left (fun a m => (a + Z.of_nat (eval_nat_mono m vars))%Z)
                   (neg_monomials l) 0%Z)%Z).
  { induction l as [| h t IH]; intro az.
    - simpl. lia.
    - (* Unfold one step of fold_left on LHS via change *)
      change (fold_left (fun a m => (a + eval_monomial m vars)%Z) (h :: t) az)
        with (fold_left (fun a m => (a + eval_monomial m vars)%Z) t
                        (az + eval_monomial h vars)%Z).
      destruct (Z.ltb_spec 0%Z (fst h)) as [Hpos | Hnonpos].
      + (* positive coefficient: goes to pos_monomials *)
        rewrite pos_step_pos, neg_step_pos by lia.
        rewrite (fold_natmono_step vars (Z.to_nat (fst h)) (snd h) (pos_monomials t) 0%Z).
        specialize (IH (az + eval_monomial h vars)%Z).
        rewrite IH.
        (* goal: az + eval_monomial h + fold_pos_t 0 - fold_neg_t 0
              = az + (0 + Z.of_nat(eval_nat_mono(Z.to_nat fh, sh)) + fold_pos_t 0) -
   fold_neg_t 0
           Use pos_mono_val to rewrite Z.of_nat(eval_nat_mono ...) -> eval_monomial h *)
        destruct h as [c exps]; simpl in *.
        rewrite pos_mono_val by lia.
        (* Both sides now have: az + eval_monomial(c,exps) + fold_pos_t 0 - fold_neg_t 0
                              = az + (eval_monomial(c,exps) + fold_pos_t 0) - fold_neg_t 0
           but LHS came from IH rewrite so it has: c * fold... form.
           Need to normalize LHS. Use `change` to rewrite c*fold as eval_monomial. *)
        change (c * fold_left (fun acc p => (acc * Z.of_nat (pow (fst p) (snd p)))%Z)
                              (combine vars exps) 1%Z)%Z
          with (eval_monomial (c, exps) vars).
        lia.
      + destruct (Z.ltb_spec (fst h) 0%Z) as [Hneg | Hzero].
        * (* negative coefficient: goes to neg_monomials *)
          rewrite pos_step_neg, neg_step_neg by lia.
          rewrite (fold_natmono_step vars (Z.to_nat (Z.opp (fst h))) (snd h) (neg_monomials t)
   0%Z).
          specialize (IH (az + eval_monomial h vars)%Z).
          rewrite IH.
          destruct h as [c exps]; simpl in *.
          rewrite neg_mono_val by lia.
          change (c * fold_left (fun acc p => (acc * Z.of_nat (pow (fst p) (snd p)))%Z)
                                (combine vars exps) 1%Z)%Z
            with (eval_monomial (c, exps) vars).
          lia.
        * (* zero coefficient: goes to neither *)
          rewrite pos_step_zero, neg_step_zero by lia.
          specialize (IH (az + eval_monomial h vars)%Z).
          rewrite IH.
          destruct h as [c exps]; simpl in *.
          (* eval_monomial (0, exps) = 0 *)
          (* eval_monomial (c, exps) = c * ... = 0 * ... = 0 since c = 0 *)
          assert (Hc : c = 0%Z) by lia. subst.
          unfold eval_monomial; simpl. lia. }
  rewrite (KEY p 0%Z). lia.
Qed.

(* ================================================================ *)
(* 9. ENCODING AND CORRECTNESS                                      *)
(* ================================================================ *)

Record h10c_inst := mk_h10c {
  h10c_ar  : nat;
```

```
  h10c_pos : NatPoly;
  h10c_neg : NatPoly;
}.

Definition H10c_sat (inst : h10c_inst) : Prop :=
  exists n,
    eval_nat_poly inst.(h10c_pos) (decode_k inst.(h10c_ar) n) =
    eval_nat_poly inst.(h10c_neg) (decode_k inst.(h10c_ar) n).

Definition poly_encode (ar : nat) (p : Poly) : h10c_inst :=
  mk_h10c ar (pos_monomials p) (neg_monomials p).

Theorem poly_encode_correct : forall ar p,
  is_solvable ar p <-> H10c_sat (poly_encode ar p).
Proof.
  intros ar p. unfold is_solvable, H10c_sat, poly_encode; simpl.
  split.
  - intros [n Hn]. exists n.
    unfold check_solution in Hn.
    destruct (Z.eq_dec (eval_poly p (decode_k ar n)) 0%Z) as [Heq | Hne].
    + rewrite eval_poly_split in Heq. lia.
    + discriminate.
  - intros [n Hn]. exists n.
    unfold check_solution.
    destruct (Z.eq_dec (eval_poly p (decode_k ar n)) 0%Z) as [Heq | Hne].
    + reflexivity.
    + exfalso. apply Hne. rewrite eval_poly_split. lia.
Qed.


(** Larchey-Wendling's undecidability theorem for h10c
    (replace with: Import from coq-library-undecidability). *)
(* ============================================================  *)
(* 6.bridge  BRIDGE: H10C_SAT -> H10c_sat (replaces LW axiom)     *)
(* ============================================================  *)

(* h10c constraint type (from coq-library-undecidability) *)
Inductive h10c : Set :=
  | h10c_one  : nat -> h10c
  | h10c_plus : nat -> nat -> nat -> h10c
  | h10c_mult : nat -> nat -> nat -> h10c.

Definition h10c_sem (c : h10c) (phi : nat -> nat) : Prop :=
  match c with
  | h10c_one x      => phi x = 1
  | h10c_plus x y z => phi x + phi y = phi z
  | h10c_mult x y z => phi x * phi y = phi z
  end.

Definition H10C_SAT (cs : list h10c) :=
  exists (phi : nat -> nat), forall c, In c cs -> h10c_sem c phi.

Axiom H10C_SAT_undec :
  ~ exists (f : list h10c -> bool),
      (forall cs, H10C_SAT cs -> f cs = true) /\
      (forall cs, ~ H10C_SAT cs -> f cs = false).


(*  1. Cantor pairing round-trip lemmas (all proved, 0 axioms)      *)
(* ================================================================ *)

Lemma even_mul_succ (s : nat) : (s*(s+1)) mod 2 = 0.
Proof.
  induction s as [|n IH]. { simpl; reflexivity. }
  replace (S n * (S n + 1)) with (n*(n+1) + 2*(S n)) by lia.
  assert (H1 : (2*S n) mod 2 = 0).
  { replace (2*S n) with (S n * 2) by lia. apply Nat.Div0.mod_mul. }
```

```
  rewrite Nat.Div0.add_mod, IH, H1. simpl. reflexivity.
Qed.

Lemma tri_2 (s : nat) : 2*(s*(s+1)/2) = s*(s+1).
Proof.
  assert (H1 := even_mul_succ s). assert (H2 := Nat.div_mod_eq (s*(s+1)) 2). lia.
Qed.

Lemma div2_sandwich (v k : nat) : 2*k+1 <= v -> v < 2*k+3 -> (v-1)/2 = k.
Proof.
  intros Hlo Hhi. assert (v = 2*k+1 \/ v = 2*k+2) by lia.
  destruct H as [-> | ->].
  - replace (2*k+1-1) with (k*2) by lia. apply Nat.div_mul. lia.
  - replace (2*k+2-1) with (k*2+1) by lia.
    rewrite Nat.div_add_l by lia. simpl. lia.
Qed.

Lemma cantor_w_pair (a b : nat) : cantor_w (cantor_pair a b) = a+b.
Proof.
  unfold cantor_w, cantor_pair. assert (Ht := tri_2 (a+b)).
  assert (Hval : 8*((a+b)*(a+b+1)/2+b)+1 = (2*(a+b)+1)*(2*(a+b)+1)+8*b) by lia.
  rewrite Hval. apply div2_sandwich.
  - apply (proj1 (Nat.sqrt_le_square _ (2*(a+b)+1))). lia.
  - apply (proj1 (Nat.sqrt_lt_square _ (2*(a+b)+3))). lia.
Qed.

Lemma cantor_snd_pair (a b : nat) : cantor_unpair_snd (cantor_pair a b) = b.
Proof.
  unfold cantor_unpair_snd, cantor_w, cantor_pair.
  assert (Ht := tri_2 (a+b)).
  assert (Hval : 8*((a+b)*(a+b+1)/2+b)+1 = (2*(a+b)+1)*(2*(a+b)+1)+8*b) by lia.
  set (v := (Nat.sqrt (8 * ((a + b) * (a + b + 1) / 2 + b) + 1) - 1) / 2).
  assert (Hw : v = a+b).
  { unfold v. rewrite Hval.
    apply div2_sandwich.
    - apply (proj1 (Nat.sqrt_le_square _ (2*(a+b)+1))). lia.
    - apply (proj1 (Nat.sqrt_lt_square _ (2*(a+b)+3))). lia. }
  rewrite Hw. lia.
Qed.

Lemma cantor_fst_pair (a b : nat) : cantor_unpair_fst (cantor_pair a b) = a.
Proof.
  unfold cantor_unpair_fst.
  assert (Hs : cantor_unpair_snd (cantor_pair a b) = b) by apply cantor_snd_pair.
  unfold cantor_unpair_snd, cantor_w, cantor_pair in *.
  assert (Ht := tri_2 (a+b)).
  assert (Hval : 8*((a+b)*(a+b+1)/2+b)+1 = (2*(a+b)+1)*(2*(a+b)+1)+8*b) by lia.
  set (v := (Nat.sqrt (8 * ((a + b) * (a + b + 1) / 2 + b) + 1) - 1) / 2) in *.
  assert (Hw : v = a+b).
  { unfold v. rewrite Hval.
    apply div2_sandwich.
    - apply (proj1 (Nat.sqrt_le_square _ (2*(a+b)+1))). lia.
    - apply (proj1 (Nat.sqrt_lt_square _ (2*(a+b)+3))). lia. }
  rewrite Hw in *. lia.
Qed.


Lemma decode_k_step (k' a b : nat) :
  decode_k (S (S k')) (cantor_pair a b) = decode_k (S k') a ++ [b].
Proof. simpl decode_k. rewrite cantor_fst_pair, cantor_snd_pair. reflexivity. Qed.

(* encode_phi phi k: witness for decode_k surjectivity *)
(* decode_k (S k') n = decode_k k' (fst n) ++ [snd n]   *)
(* so encode stores phi(k-1) as the last (snd) element   *)
Fixpoint encode_phi (phi : nat -> nat) (k : nat) : nat :=
  match k with
  | 0    => 0
```

```
  | S O  => phi O
  | S k' => cantor_pair (encode_phi phi k') (phi k')
  end.

Lemma encode_phi_ss (phi : nat -> nat) (k'' : nat) :
  encode_phi phi (S (S k'')) = cantor_pair (encode_phi phi (S k'')) (phi (S k'')).
Proof. simpl. destruct k''; reflexivity. Qed.

Lemma map_seq_snoc (phi : nat -> nat) (n : nat) :
  map phi (seq 0 n) ++ [phi n] = map phi (seq 0 (S n)).
Proof. rewrite seq_S, map_app. reflexivity. Qed.

Lemma decode_encode_phi : forall k phi,
  decode_k k (encode_phi phi k) = map phi (seq 0 k).
Proof.
  intro k. induction k as [|k' IH]; intro phi. { simpl. reflexivity. }
  destruct k' as [|k'']. { simpl. reflexivity. }
  rewrite encode_phi_ss, decode_k_step, (IH phi). apply map_seq_snoc.
Qed.

Lemma decode_k_surjective (k : nat) (phi : nat -> nat) :
  exists n, forall i, i < k -> nth i (decode_k k n) 0 = phi i.
Proof.
  exists (encode_phi phi k). intros i Hi.
  rewrite decode_encode_phi.
  assert (Hlen_map : length (map phi (seq 0 k)) = k).
  { rewrite length_map, length_seq. reflexivity. }
  rewrite nth_indep with (d' := phi i) by lia.
  rewrite map_nth, seq_nth by lia. reflexivity.
Qed.

(* ================================================================ *)
(*  2. Polynomial arithmetic                                        *)
(* ================================================================ *)

Fixpoint map2_add (e1 e2 : list nat) : list nat :=
  match e1, e2 with x::xs, y::ys => (x+y) :: map2_add xs ys | _, _ => [] end.

Definition mono_mult (m1 m2 : NatMonomial) : NatMonomial :=
  (fst m1 * fst m2, map2_add (snd m1) (snd m2)).
Definition poly_mult (p1 p2 : NatPoly) : NatPoly :=
  flat_map (fun m1 => map (mono_mult m1) p2) p1.

Lemma poly_mult_cons (m : NatMonomial) (p1 p2 : NatPoly) :
  poly_mult (m :: p1) p2 = map (mono_mult m) p2 ++ poly_mult p1 p2.
Proof. unfold poly_mult. simpl. reflexivity. Qed.

Definition poly_scale (c : nat) (p : NatPoly) : NatPoly :=
  map (fun m => (c * fst m, snd m)) p.
Definition poly_sq (p : NatPoly) := poly_mult p p.

Lemma pow_add (v a b : nat) : pow v (a+b) = pow v a * pow v b.
Proof. induction a as [|a' IH]. simpl; lia. simpl. rewrite IH. lia. Qed.

Lemma eval_aux_scale (exps vals : list nat) (a c : nat) :
  eval_nat_mono_aux exps vals (a * c) = a * eval_nat_mono_aux exps vals c.
Proof.
  revert vals a c. induction exps as [|e es IH]; intros vals a c. { simpl. lia. }
  destruct vals as [|v vs]. { simpl. lia. }
  simpl. rewrite <- IH. f_equal. lia.
Qed.

Lemma eval_aux_mul_exp : forall (e1 e2 vars : list nat),
  length e1 = length vars -> length e2 = length vars ->
  forall (a1 a2 : nat),
  eval_nat_mono_aux (map2_add e1 e2) vars (a1 * a2) =
  eval_nat_mono_aux e1 vars a1 * eval_nat_mono_aux e2 vars a2.
```

```
Proof.
  intro e1. induction e1 as [|x xs IH].
  - intros e2 vars Hl1 Hl2 a1 a2.
    destruct vars; [|simpl in Hl1; discriminate].
    destruct e2; [|simpl in Hl2; discriminate]. simpl. lia.
  - intros e2 vars Hl1 Hl2 a1 a2.
    destruct vars as [|v vs]; [simpl in Hl1; discriminate|].
    destruct e2 as [|y ys]; [simpl in Hl2; discriminate|].
    simpl in Hl1, Hl2. injection Hl1 as Hl1. injection Hl2 as Hl2.
    simpl map2_add. simpl eval_nat_mono_aux. rewrite pow_add.
    replace (a1 * a2 * (pow v x * pow v y))
       with ((a1 * pow v x) * (a2 * pow v y)) by lia.
    apply IH; assumption.
Qed.

Lemma map2_add_length (e1 e2 : list nat) (H : length e1 = length e2) :
  length (map2_add e1 e2) = length e1.
Proof.
  revert e2 H. induction e1 as [|x xs IH]; intros e2 H.
  - simpl. reflexivity.
  - destruct e2 as [|y ys]; [simpl in H; discriminate|].
    simpl in H. injection H as H. simpl. rewrite IH by exact H. reflexivity.
Qed.

Lemma eval_mono_mult (m1 m2 : NatMonomial) (vars : list nat) :
  length (snd m1) = length vars -> length (snd m2) = length vars ->
  eval_nat_mono (mono_mult m1 m2) vars =
  eval_nat_mono m1 vars * eval_nat_mono m2 vars.
Proof.
  intros Hl1 Hl2. unfold mono_mult, eval_nat_mono. simpl.
  assert (Heq : eval_nat_mono_aux (map2_add (snd m1) (snd m2)) vars 1 =
                eval_nat_mono_aux (snd m1) vars 1 * eval_nat_mono_aux (snd m2) vars 1).
  { replace 1 with (1*1) at 1 by lia. apply eval_aux_mul_exp; assumption. }
  rewrite Heq. lia.
Qed.

Lemma fold_left_add_acc (p : NatPoly) (vars : list nat) (a : nat) :
  fold_left (fun acc m => acc + eval_nat_mono m vars) p a =
  a + fold_left (fun acc m => acc + eval_nat_mono m vars) p 0.
Proof.
  revert a. induction p as [|m p' IH]; intro a. { simpl. lia. }
  simpl. rewrite (IH (a + _)), (IH (eval_nat_mono m vars)). lia.
Qed.

Lemma eval_poly_cons (m : NatMonomial) (p : NatPoly) (vars : list nat) :
  eval_nat_poly (m :: p) vars = eval_nat_mono m vars + eval_nat_poly p vars.
Proof. unfold eval_nat_poly. simpl. rewrite fold_left_add_acc. lia. Qed.

Lemma eval_poly_nil (vars : list nat) : eval_nat_poly [] vars = 0.
Proof. reflexivity. Qed.

Lemma eval_poly_app (p1 p2 : NatPoly) (vars : list nat) :
  eval_nat_poly (p1 ++ p2) vars = eval_nat_poly p1 vars + eval_nat_poly p2 vars.
Proof.
  induction p1 as [|m p1' IH]. { simpl. reflexivity. }
  rewrite <- app_comm_cons, !eval_poly_cons, IH. lia.
Qed.

Lemma eval_scaled_mono (c : nat) (m : NatMonomial) (vars : list nat) :
  eval_nat_mono (c * fst m, snd m) vars = c * eval_nat_mono m vars.
Proof. unfold eval_nat_mono. simpl. lia. Qed.

Lemma eval_poly_scale (c : nat) (p : NatPoly) (vars : list nat) :
  eval_nat_poly (poly_scale c p) vars = c * eval_nat_poly p vars.
Proof.
  induction p as [|m p' IH].
  - unfold eval_nat_poly, poly_scale. simpl. ring.
```

```
  - unfold poly_scale. simpl map. rewrite !eval_poly_cons, eval_scaled_mono.
    fold (poly_scale c p'). rewrite IH. ring.
Qed.

Lemma eval_poly_mult_inner (m : NatMonomial) (p : NatPoly) (vars : list nat) :
  length (snd m) = length vars ->
  (forall m', In m' p -> length (snd m') = length vars) ->
  eval_nat_poly (map (mono_mult m) p) vars =
  eval_nat_mono m vars * eval_nat_poly p vars.
Proof.
  intros Hm Hp. induction p as [|m' p' IH]. { unfold eval_nat_poly. simpl. lia. }
  simpl. rewrite !eval_poly_cons.
  assert (Hip : forall m'', In m'' p' -> length (snd m'') = length vars).
  { intros m'' Hm''. apply Hp. right. exact Hm''. }
  rewrite (IH Hip).
  assert (Hml : length (snd m) = length vars) by exact Hm.
  assert (Hm'l : length (snd m') = length vars) by (apply Hp; left; reflexivity).
  rewrite eval_mono_mult by assumption. lia.
Qed.

Lemma eval_poly_mult_correct (p1 p2 : NatPoly) (vars : list nat) :
  (forall m, In m p1 -> length (snd m) = length vars) ->
  (forall m, In m p2 -> length (snd m) = length vars) ->
  eval_nat_poly (poly_mult p1 p2) vars =
  eval_nat_poly p1 vars * eval_nat_poly p2 vars.
Proof.
  intros Hp1 Hp2. induction p1 as [|m p1' IH]. { unfold eval_nat_poly. simpl. lia. }
  rewrite poly_mult_cons, eval_poly_app.
  assert (Hm_in : length (snd m) = length vars) by (apply Hp1; left; reflexivity).
  rewrite eval_poly_mult_inner by (exact Hm_in || exact Hp2).
  rewrite eval_poly_cons.
  assert (Hp1' : forall m', In m' p1' -> length (snd m') = length vars).
  { intros m' Hm'. apply Hp1. right. exact Hm'. }
  rewrite IH by exact Hp1'. lia.
Qed.

Lemma eval_poly_sq (p : NatPoly) (vars : list nat) :
  (forall m, In m p -> length (snd m) = length vars) ->
  eval_nat_poly (poly_sq p) vars = eval_nat_poly p vars * eval_nat_poly p vars.
Proof. intro H. apply eval_poly_mult_correct; exact H. Qed.

(* ================================================================ *)
(*  3. Monomial length preservation                                 *)
(* ================================================================ *)

Definition all_len (k : nat) (p : NatPoly) : Prop :=
  forall m, In m p -> length (snd m) = k.

Lemma all_len_nil (k : nat) : all_len k [].
Proof. intros m []. Qed.

Lemma all_len_app (k : nat) (p1 p2 : NatPoly) :
  all_len k p1 -> all_len k p2 -> all_len k (p1 ++ p2).
Proof.
  intros H1 H2 m Hm. apply in_app_iff in Hm.
  destruct Hm; [apply H1 | apply H2]; assumption.
Qed.

Lemma all_len_mono_mult (k : nat) (m1 m2 : NatMonomial) :
  length (snd m1) = k -> length (snd m2) = k ->
  length (snd (mono_mult m1 m2)) = k.
Proof.
  intros Hl1 Hl2. unfold mono_mult. simpl.
  assert (Hlen_eq : length (snd m1) = length (snd m2)) by (rewrite Hl1, Hl2; reflexivity).
  rewrite map2_add_length by exact Hlen_eq. exact Hl1.
Qed.
```

```
Lemma all_len_mult (k : nat) (p1 p2 : NatPoly) :
  all_len k p1 -> all_len k p2 -> all_len k (poly_mult p1 p2).
Proof.
  intros H1 H2 m Hm. unfold poly_mult in Hm.
  apply in_flat_map in Hm. destruct Hm as [m1 [Hm1 Hm']].
  apply in_map_iff in Hm'. destruct Hm' as [m2 [<- Hm2]].
  apply all_len_mono_mult; [apply H1; exact Hm1 | apply H2; exact Hm2].
Qed.

Lemma all_len_scale (k c : nat) (p : NatPoly) :
  all_len k p -> all_len k (poly_scale c p).
Proof.
  intros H m Hm. unfold poly_scale in Hm. apply in_map_iff in Hm.
  destruct Hm as [m' [<- Hm']]. simpl. apply H. exact Hm'.
Qed.

Lemma all_len_sq (k : nat) (p : NatPoly) :
  all_len k p -> all_len k (poly_sq p).
Proof. intro H. apply all_len_mult; exact H. Qed.

(* ================================================================ *)
(*  4. h10c constraint encoding                                     *)
(* ================================================================ *)

(** unit_exp k i: exponent vector with 1 at position i, 0 elsewhere *)
Definition unit_exp (k i : nat) : list nat :=
  map (fun j => if Nat.eqb j i then 1 else 0) (seq 0 k).

Lemma unit_exp_length (k i : nat) : length (unit_exp k i) = k.
Proof. unfold unit_exp. rewrite length_map, length_seq. reflexivity. Qed.

(** eval_aux over unit_exp gives vars[i] *)
Lemma eval_aux_all_zero : forall exps vals acc,
  (forall e, In e exps -> e = 0) -> eval_nat_mono_aux exps vals acc = acc.
Proof.
  induction exps as [|e es IH]; intros vals acc Hall. { simpl; reflexivity. }
  destruct vals as [|v vs]. { simpl; reflexivity. }
  simpl. assert (e = 0) by (apply Hall; left; reflexivity). subst.
  simpl pow. rewrite Nat.mul_1_r. apply IH.
  intros e' He'. apply Hall. right. exact He'.
Qed.

Lemma eval_aux_unit : forall k i vars acc,
  length vars = k -> i < k ->
  eval_nat_mono_aux (unit_exp k i) vars acc = acc * nth i vars 0.
Proof.
  induction k as [|k' IH]; intros i vars acc Hlen Hi. { lia. }
  destruct vars as [|v vs]; [simpl in Hlen; discriminate|].
  simpl in Hlen; injection Hlen as Hlen'.
  unfold unit_exp; simpl (seq 0 (S k')); simpl (map _ (0 :: _)); simpl eval_nat_mono_aux.
  destruct i as [|i'].
  - simpl Nat.eqb; simpl pow; simpl (nth 0 (v::vs) 0).
    rewrite eval_aux_all_zero; [lia|].
    intros e He. apply in_map_iff in He. destruct He as [j [<- Hj]].
    apply in_seq in Hj. simpl.
    destruct (Nat.eqb j 0) eqn:E; [apply Nat.eqb_eq in E; lia | reflexivity].
  - simpl (Nat.eqb 0 (S i')); simpl pow; rewrite Nat.mul_1_r.
    simpl (nth (S i') (v::vs) 0).
    assert (Hm : map (fun j => if Nat.eqb j (S i') then 1 else 0) (seq 1 k') = unit_exp k' i').
    { unfold unit_exp. rewrite <- seq_shift, map_map. apply map_ext. intro j. simpl.
   reflexivity. }
    rewrite Hm.
    assert (IHapp : eval_nat_mono_aux (unit_exp k' i') vs acc = acc * nth i' vs 0).
    { apply IH; [exact Hlen' | lia]. }
    rewrite IHapp. lia.
Qed.
```

```
Definition var_mono (k i : nat) : NatMonomial := (1, unit_exp k i).

Lemma var_mono_len (k i : nat) : length (snd (var_mono k i)) = k.
Proof. apply unit_exp_length. Qed.

Lemma eval_var_mono (k i : nat) (vars : list nat) :
  length vars = k -> i < k ->
  eval_nat_mono (var_mono k i) vars = nth i vars 0.
Proof.
  intros Hlen Hi. unfold var_mono, eval_nat_mono. simpl.
  rewrite eval_aux_unit by assumption. lia.
Qed.

(** mul_exp k i j: exponent vector with 1 at i and 1 at j (2 at i if i=j) *)
Definition mul_exp (k i j : nat) : list nat :=
  map (fun p => (if Nat.eqb p i then 1 else 0) +
                (if Nat.eqb p j then 1 else 0)) (seq 0 k).

Lemma mul_exp_length (k i j : nat) : length (mul_exp k i j) = k.
Proof. unfold mul_exp. rewrite length_map, length_seq. reflexivity. Qed.

Lemma eval_aux_mul_exp2 : forall k i j vars acc,
  length vars = k -> i < k -> j < k ->
  eval_nat_mono_aux (mul_exp k i j) vars acc =
  acc * nth i vars 0 * nth j vars 0.
Proof.
  induction k as [|k' IH]; intros i j vars acc Hlen Hi Hj. { lia. }
  destruct vars as [|v vs]; [simpl in Hlen; discriminate|].
  simpl in Hlen; injection Hlen as Hlen'.
  unfold mul_exp; simpl (seq 0 (S k')); simpl (map _ (0 :: _)); simpl eval_nat_mono_aux.
  destruct i as [|i']; destruct j as [|j'].
  - simpl; simpl pow.
    rewrite eval_aux_all_zero; [lia|].
    intros e He. apply in_map_iff in He. destruct He as [p [<- Hp]].
    apply in_seq in Hp. simpl.
    destruct (Nat.eqb p 0) eqn:E; [apply Nat.eqb_eq in E; lia | simpl; reflexivity].
  - simpl (Nat.eqb 0 0); simpl (Nat.eqb 0 (S j')); simpl pow.
    simpl (nth 0 (v::vs) 0); simpl (nth (S j') (v::vs) 0).
    assert (Hm : map (fun p => (if Nat.eqb p 0 then 1 else 0) +
                               (if Nat.eqb p (S j') then 1 else 0)) (seq 1 k') = unit_exp k' j
  ').
    { unfold unit_exp. rewrite <- seq_shift, map_map. apply map_ext. intro p. simpl. lia. }
    rewrite Hm. rewrite eval_aux_unit by (assumption || lia). lia.
  - simpl (Nat.eqb 0 (S i')); simpl (Nat.eqb 0 0); simpl pow.
    simpl (nth (S i') (v::vs) 0); simpl (nth 0 (v::vs) 0).
    assert (Hm : map (fun p => (if Nat.eqb p (S i') then 1 else 0) +
                               (if Nat.eqb p 0 then 1 else 0)) (seq 1 k') = unit_exp k' i').
    { unfold unit_exp. rewrite <- seq_shift, map_map. apply map_ext. intro p. simpl. lia. }
    rewrite Hm. rewrite eval_aux_unit by (assumption || lia). lia.
  - simpl (Nat.eqb 0 (S i')); simpl (Nat.eqb 0 (S j')); simpl pow. rewrite Nat.mul_1_r.
    simpl (nth (S i') (v::vs) 0); simpl (nth (S j') (v::vs) 0).
    assert (Hm : map (fun p => (if Nat.eqb p (S i') then 1 else 0) +
                               (if Nat.eqb p (S j') then 1 else 0)) (seq 1 k') = mul_exp k' i'
  j').
    { unfold mul_exp. rewrite <- seq_shift, map_map. apply map_ext. intro p. simpl. reflexivity
  . }
    rewrite Hm.
    assert (IHapp : eval_nat_mono_aux (mul_exp k' i' j') vs acc = acc * nth i' vs 0 * nth j' vs
   0).
    { apply IH; [exact Hlen' | lia | lia]. }
    rewrite IHapp. lia.
Qed.

Definition mul_mono (k i j : nat) : NatMonomial := (1, mul_exp k i j).

Lemma mul_mono_len (k i j : nat) : length (snd (mul_mono k i j)) = k.
Proof. apply mul_exp_length. Qed.
```

```
Lemma eval_mul_mono (k i j : nat) (vars : list nat) :
  length vars = k -> i < k -> j < k ->
  eval_nat_mono (mul_mono k i j) vars = nth i vars 0 * nth j vars 0.
Proof.
  intros Hlen Hi Hj. unfold mul_mono, eval_nat_mono. simpl.
  rewrite eval_aux_mul_exp2 by assumption. lia.
Qed.

(** const_one k: the constant polynomial 1 *)
Definition const_one_exp (k : nat) : list nat := map (fun _ => 0) (seq 0 k).

Lemma const_one_exp_length (k : nat) : length (const_one_exp k) = k.
Proof. unfold const_one_exp. rewrite length_map, length_seq. reflexivity. Qed.

Definition const_one (k : nat) : NatMonomial := (1, const_one_exp k).

Lemma eval_const_one (k : nat) (vars : list nat) (Hlen : length vars = k) :
  eval_nat_mono (const_one k) vars = 1.
Proof.
  unfold const_one, eval_nat_mono. simpl.
  rewrite eval_aux_all_zero; [lia|].
  intros e He. apply in_map_iff in He. destruct He as [_ [<- _]]. reflexivity.
Qed.

(** Encode one h10c constraint as (pos, neg) NatPoly pair *)
Definition encode_c (k : nat) (c : h10c) : NatPoly * NatPoly :=
  match c with
  | h10c_one x      => ([const_one k], [var_mono k x])
  | h10c_plus x y z => ([var_mono k x; var_mono k y], [var_mono k z])
  | h10c_mult x y z => ([mul_mono k x y], [var_mono k z])
  end.

Definition h10c_max_var (c : h10c) : nat :=
  match c with
  | h10c_one x      => x
  | h10c_plus x y z => Nat.max x (Nat.max y z)
  | h10c_mult x y z => Nat.max x (Nat.max y z)
  end.

Lemma encode_c_all_len (k : nat) (c : h10c) (Hk : h10c_max_var c < k) :
  all_len k (fst (encode_c k c)) /\ all_len k (snd (encode_c k c)).
Proof.
  destruct c; simpl in Hk; simpl.
  - split; intros m Hm.
    + destruct Hm as [<- | []]. apply const_one_exp_length.
    + destruct Hm as [<- | []]. apply var_mono_len.
  - split; intros m Hm.
    + destruct Hm as [<- | [<- | []]]; apply var_mono_len.
    + destruct Hm as [<- | []]. apply var_mono_len.
  - split; intros m Hm.
    + destruct Hm as [<- | []]. apply mul_mono_len.
    + destruct Hm as [<- | []]. apply var_mono_len.
Qed.

(** Semantic correctness: (pos eval = neg eval) <-> h10c_sem c phi *)
Lemma encode_c_correct (k : nat) (c : h10c) (Hk : h10c_max_var c < k)
    (vars : list nat) (Hlen : length vars = k) :
  eval_nat_poly (fst (encode_c k c)) vars =
  eval_nat_poly (snd (encode_c k c)) vars
  <->
  h10c_sem c (fun i => nth i vars 0).
Proof.
  destruct c; simpl in Hk; simpl (encode_c _ _); simpl h10c_sem;
    simpl fst; simpl snd.
  - (* h10c_one x: 1 = phi(x) *)
    rewrite !eval_poly_cons, !eval_poly_nil.
```

```
    rewrite eval_const_one by exact Hlen.
    rewrite eval_var_mono by (exact Hlen || lia).
    split; lia.
  - (* h10c_plus x y z: phi(x)+phi(y) = phi(z) *)
    rewrite !eval_poly_cons, !eval_poly_nil.
    rewrite !eval_var_mono by (exact Hlen || lia).
    split; lia.
  - (* h10c_mult x y z: phi(x)*phi(y) = phi(z) *)
    rewrite !eval_poly_cons, !eval_poly_nil.
    rewrite eval_mul_mono by (exact Hlen || lia).
    rewrite eval_var_mono by (exact Hlen || lia).
    split; lia.
Qed.

(* ================================================================ *)
(*  5. Conjunction encoding via sum of squares                      *)
(* ================================================================ *)

Lemma sq_sum_iff_eq (a b : nat) : a * a + b * b = 2 * a * b <-> a = b.
Proof.
  split.
  - intro H.
    destruct (Nat.le_gt_cases a b) as [Hab | Hab].
    + assert (Hd : (b - a) * (b - a) + 2 * a * b = b * b + a * a).
      { remember (b - a) as d. assert (Hbd : b = d + a) by lia. subst b. ring. }
      assert (Hk : (b - a) * (b - a) = 0) by lia.
      destruct (b - a) as [|n] eqn:Hba.
      * lia.
      * exfalso. simpl in Hk. lia.
    + assert (Hd : (a - b) * (a - b) + 2 * a * b = a * a + b * b).
      { remember (a - b) as d. assert (Had : a = d + b) by lia. subst a. ring. }
      assert (Hk : (a - b) * (a - b) = 0) by lia.
      destruct (a - b) as [|n] eqn:Hba.
      * lia.
      * exfalso. simpl in Hk. lia.
  - intro Heq. subst. lia.
Qed.

Lemma sq_ge (a b : nat) : a * a + b * b >= 2 * a * b.
Proof.
  destruct (Nat.le_gt_cases a b) as [Hle | Hlt].
  - assert (Hd : (b - a) * (b - a) + 2 * a * b = b * b + a * a).
    { remember (b - a) as d. assert (Hbd : b = d + a) by lia. subst b. ring. }
    lia.
  - assert (Hd : (a - b) * (a - b) + 2 * a * b = a * a + b * b).
    { remember (a - b) as d. assert (Had : a = d + b) by lia. subst a. ring. }
    lia.
Qed.

Lemma sq_eq_zero (a b rv sv : nat)
    (H : a*a + (b*b + rv) = 2*(a*b) + sv) (Hge : rv >= sv) : a = b.
Proof.
  assert (Hge2 : a*a + b*b >= 2*a*b) by apply sq_ge.
  assert (Heq : a*a + b*b = 2*a*b) by lia.
  destruct (Nat.le_gt_cases a b) as [Hle | Hlt].
  - assert (Hd : (b - a) * (b - a) + 2 * a * b = b * b + a * a).
    { remember (b - a) as d. assert (Hbd : b = d + a) by lia. subst b. ring. }
    assert (Hk : (b - a) * (b - a) = 0) by lia.
    destruct (b - a) as [|n] eqn:Hba.
    * lia.
    * exfalso. simpl in Hk. lia.
  - assert (Hd : (a - b) * (a - b) + 2 * a * b = a * a + b * b).
    { remember (a - b) as d. assert (Had : a = d + b) by lia. subst a. ring. }
    assert (Hk : (a - b) * (a - b) = 0) by lia.
    destruct (a - b) as [|n] eqn:Hba.
    * lia.
    * exfalso. simpl in Hk. lia.
```

```
Qed.

Definition compat (p : NatPoly) (vars : list nat) :=
  forall m, In m p -> length (snd m) = length vars.


(** Fold list of pairs into one (pos, neg) via sum-of-squares *)
Fixpoint conj_encode (pairs : list (NatPoly * NatPoly)) : NatPoly * NatPoly :=
  match pairs with
  | [] => ([], [])
  | (p, n) :: rest =>
      let (rp, rn) := conj_encode rest in
      (poly_sq p ++ poly_sq n ++ rp,
       poly_scale 2 (poly_mult p n) ++ rn)
  end.

Lemma conj_pos_ge_neg (pairs : list (NatPoly * NatPoly)) (vars : list nat)
  (HC : forall pr, In pr pairs -> compat (fst pr) vars /\ compat (snd pr) vars) :
  eval_nat_poly (fst (conj_encode pairs)) vars >=
  eval_nat_poly (snd (conj_encode pairs)) vars.
Proof.
  induction pairs as [|[lp ln] rest IH]. simpl; lia.
  simpl conj_encode. destruct (conj_encode rest) as [rp rn].
  simpl fst. simpl snd.
  assert (Hpn := HC (lp,ln) (or_introl eq_refl)). destruct Hpn as [Hlp Hln].
  simpl fst in Hlp. simpl snd in Hln. unfold compat in Hlp, Hln.
  assert (Hpsq : eval_nat_poly (poly_sq lp) vars =
                 eval_nat_poly lp vars * eval_nat_poly lp vars).
  { apply eval_poly_sq. exact Hlp. }
  assert (Hnsq : eval_nat_poly (poly_sq ln) vars =
                 eval_nat_poly ln vars * eval_nat_poly ln vars).
  { apply eval_poly_sq. exact Hln. }
  assert (Hmn : eval_nat_poly (poly_mult lp ln) vars =
                eval_nat_poly lp vars * eval_nat_poly ln vars).
  { apply eval_poly_mult_correct; [exact Hlp | exact Hln]. }
  rewrite !eval_poly_app, eval_poly_scale, Hpsq, Hnsq, Hmn.
  set (a := eval_nat_poly lp vars). set (b := eval_nat_poly ln vars).
  set (rp_v := eval_nat_poly rp vars). set (rn_v := eval_nat_poly rn vars).
  assert (IHok : forall pr, In pr rest -> compat (fst pr) vars /\ compat (snd pr) vars).
  { intros pr Hpr. apply HC. right. exact Hpr. }
  pose proof (IH IHok) as Hge. simpl fst in Hge. simpl snd in Hge.
  fold rp_v rn_v in Hge.
  pose proof (sq_ge a b). lia.
Qed.

Lemma conj_encode_correct (k : nat) (pairs : list (NatPoly * NatPoly)) (vars : list nat)
  (Hlen : length vars = k)
  (Hall : forall pr, In pr pairs ->
    all_len k (fst pr) /\ all_len k (snd pr)) :
  eval_nat_poly (fst (conj_encode pairs)) vars =
  eval_nat_poly (snd (conj_encode pairs)) vars
  <->
  forall pr, In pr pairs ->
    eval_nat_poly (fst pr) vars = eval_nat_poly (snd pr) vars.
Proof.
  assert (HC : forall pr, In pr pairs -> compat (fst pr) vars /\ compat (snd pr) vars).
  { intros pr Hpr. destruct (Hall pr Hpr) as [Hf Hs]. split.
    - intros m Hm. rewrite Hlen. apply Hf. exact Hm.
    - intros m Hm. rewrite Hlen. apply Hs. exact Hm. }
  induction pairs as [|[lp ln] rest IH].
  - simpl. split. intros _ pr []. intros; lia.
  - simpl conj_encode. destruct (conj_encode rest) as [rp rn] eqn:Hrest.
    simpl fst. simpl snd.
    assert (Hpn := HC (lp,ln) (or_introl eq_refl)). destruct Hpn as [Hlp Hln].
    simpl fst in Hlp. simpl snd in Hln. unfold compat in Hlp, Hln.
    assert (Hpsq : eval_nat_poly (poly_sq lp) vars =
                   eval_nat_poly lp vars * eval_nat_poly lp vars).
```

```
    { apply eval_poly_sq. exact Hlp. }
    assert (Hnsq : eval_nat_poly (poly_sq ln) vars =
                   eval_nat_poly ln vars * eval_nat_poly ln vars).
    { apply eval_poly_sq. exact Hln. }
    assert (Hmn : eval_nat_poly (poly_mult lp ln) vars =
                  eval_nat_poly lp vars * eval_nat_poly ln vars).
    { apply eval_poly_mult_correct; [exact Hlp | exact Hln]. }
    assert (HCR : forall pr, In pr rest -> compat (fst pr) vars /\ compat (snd pr) vars).
    { intros pr Hpr. apply HC. right. exact Hpr. }
    assert (HallR : forall pr, In pr rest -> all_len k (fst pr) /\ all_len k (snd pr)).
    { intros pr Hpr. apply Hall. right. exact Hpr. }
    (* IH has (rp,rn) already from destruct. Use simpl fst/snd to match. *)
    assert (IH' : eval_nat_poly rp vars = eval_nat_poly rn vars <->
                  forall pr, In pr rest ->
                    eval_nat_poly (fst pr) vars = eval_nat_poly (snd pr) vars).
    { assert (IHraw := IH HallR HCR).
      simpl fst in IHraw. simpl snd in IHraw. exact IHraw. }
    clear IH.
    assert (Hge_rest : eval_nat_poly rp vars >= eval_nat_poly rn vars).
    { assert (Hraw : eval_nat_poly (fst (conj_encode rest)) vars >=
                     eval_nat_poly (snd (conj_encode rest)) vars).
      { apply conj_pos_ge_neg. exact HCR. }
      rewrite Hrest in Hraw. simpl fst in Hraw. simpl snd in Hraw. exact Hraw. }
    rewrite !eval_poly_app, eval_poly_scale, Hpsq, Hnsq, Hmn.
    set (a := eval_nat_poly lp vars). set (b := eval_nat_poly ln vars).
    set (rv := eval_nat_poly rp vars). set (sv := eval_nat_poly rn vars).
    fold rv sv in IH'. fold rv sv in Hge_rest.
    assert (Hge2 : a*a + b*b >= 2*a*b) by apply sq_ge.
    split.
    + intro Heq. intros [hp hn] [Hin_head | Hin_rest].
      * injection Hin_head as -> ->. simpl.
        eapply sq_eq_zero; [exact Heq | exact Hge_rest].
      * apply IH'. lia. exact Hin_rest.
    + intro Hgoal.
      assert (Ha : a = b) by exact (Hgoal (lp,ln) (or_introl eq_refl)).
      subst a.
      assert (Hrvsv : rv = sv).
      { apply IH'. intros pr Hpr. apply Hgoal. right. exact Hpr. }
      rewrite Hrvsv. lia.
Qed.

(* ================================================================ *)
(*  6. The reduction: H10C_SAT -> H10c_sat                          *)
(* ================================================================ *)

Definition list_max_var (cs : list h10c) : nat :=
  fold_left (fun acc c => Nat.max acc (h10c_max_var c)) cs 0.

Lemma fold_max_ge_acc (cs : list h10c) (a : nat) :
  fold_left (fun acc c => Nat.max acc (h10c_max_var c)) cs a >= a.
Proof.
  revert a. induction cs as [|c cs' IH]; intro a. { simpl; lia. }
  simpl. specialize (IH (Nat.max a (h10c_max_var c))). lia.
Qed.

Lemma fold_max_mono_acc (cs : list h10c) (a b : nat) (H : a >= b) :
  fold_left (fun acc c => Nat.max acc (h10c_max_var c)) cs a >=
  fold_left (fun acc c => Nat.max acc (h10c_max_var c)) cs b.
Proof.
  revert a b H. induction cs as [|c cs' IH]; intros a b H. { simpl; exact H. }
  simpl. apply IH. lia.
Qed.

Lemma list_max_var_bound (c : h10c) (cs : list h10c) :
  In c cs -> h10c_max_var c < list_max_var cs + 1.
Proof.
  intro Hc. unfold list_max_var.
```

```
  induction cs as [|c' cs' IH]. { destruct Hc. }
  simpl fold_left. destruct Hc as [Heq | Hc].
  - subst c'.
    pose proof (fold_max_ge_acc cs' (h10c_max_var c)). lia.
  - specialize (IH Hc).
    assert (Hmono : fold_left (fun acc c => Nat.max acc (h10c_max_var c)) cs' (h10c_max_var c')
    >=
                    fold_left (fun acc c => Nat.max acc (h10c_max_var c)) cs' 0).
    { apply fold_max_mono_acc. lia. }
    lia.
Qed.

Lemma decode_k_length : forall ar n, length (decode_k ar n) = ar.
Proof.
  intro ar. induction ar as [|ar' IH]; intro n.
  - simpl. reflexivity.
  - destruct ar' as [|ar''].
    + simpl. reflexivity.
    + change (decode_k (S (S ar'')) n) with
        (decode_k (S ar'') (cantor_unpair_fst n) ++ [cantor_unpair_snd n]).
      rewrite length_app. simpl (length [_]).
      assert (H := IH (cantor_unpair_fst n)). lia.
Qed.

Definition reduce (cs : list h10c) : h10c_inst :=
  let k := list_max_var cs + 1 in
  let pairs := map (fun c => encode_c k c) cs in
  let (pos, neg) := conj_encode pairs in
  mk_h10c k pos neg.

Theorem h10c_sat_reduction_exists :
  exists (reduce : list h10c -> h10c_inst),
  forall cs, H10C_SAT cs <-> H10c_sat (reduce cs).
Proof.
  exists reduce. intro cs.
  unfold H10c_sat, reduce.
  set (k := list_max_var cs + 1).
  set (pairs := map (fun c => encode_c k c) cs).
  destruct (conj_encode pairs) as [pos neg] eqn:Hpair.
  simpl (h10c_ar _). simpl (h10c_pos _). simpl (h10c_neg _).
  split.
  - (* H10C_SAT cs -> H10c_sat *)
    unfold H10C_SAT. intros [phi Hphi].
    destruct (decode_k_surjective k phi) as [n Hn].
    exists n.
    assert (Hlen : length (decode_k k n) = k) by apply decode_k_length.
    set (vars := decode_k k n).
    assert (Hphi_vars : forall i, i < k -> phi i = nth i vars 0).
    { intros i Hi. unfold vars. rewrite <- Hn by exact Hi. reflexivity. }
    assert (Henc : forall c, In c cs ->
        eval_nat_poly (fst (encode_c k c)) vars =
        eval_nat_poly (snd (encode_c k c)) vars).
    { intros c Hc.
      eapply encode_c_correct; [apply list_max_var_bound; exact Hc | exact Hlen |].
      assert (Hphic := Hphi c Hc).
      assert (Hcmax : h10c_max_var c < k) by (apply list_max_var_bound; exact Hc).
      destruct c; simpl in *; rename Hphic into Hc'.
      - rewrite <- Hphi_vars by lia. exact Hc'.
      - rewrite <- !Hphi_vars by lia. exact Hc'.
      - rewrite <- !Hphi_vars by lia. exact Hc'. }
    assert (Hfst_pos : fst (conj_encode pairs) = pos).
    { rewrite Hpair. reflexivity. }
    assert (Hsnd_neg : snd (conj_encode pairs) = neg).
    { rewrite Hpair. reflexivity. }
    assert (HHall : forall pr, In pr pairs -> all_len k (fst pr) /\ all_len k (snd pr)).
    { intros [p q] Hpq. apply in_map_iff in Hpq.
      destruct Hpq as [c [<- Hc]]. apply encode_c_all_len.
```

```
      apply list_max_var_bound. exact Hc. }
    rewrite <- Hfst_pos, <- Hsnd_neg.
    pose proof (@conj_encode_correct k pairs vars Hlen HHall) as Hiff.
    apply Hiff.
    intros [p q] Hpq. apply in_map_iff in Hpq.
    destruct Hpq as [c [<- Hc]]. apply Henc. exact Hc.
  - (* H10c_sat -> H10C_SAT *)
    intros [n Hn].
    set (vars := decode_k k n).
    assert (Hlen : length vars = k) by (unfold vars; apply decode_k_length).
    unfold H10C_SAT.
    exists (fun i => nth i vars 0).
    intros c Hc.
    assert (Hfst_pos : fst (conj_encode pairs) = pos) by (rewrite Hpair; reflexivity).
    assert (Hsnd_neg : snd (conj_encode pairs) = neg) by (rewrite Hpair; reflexivity).
    assert (Hall := Hn).
    assert (HHallR : forall pr, In pr pairs -> all_len k (fst pr) /\ all_len k (snd pr)).
    { intros [p q] Hpq. apply in_map_iff in Hpq.
      destruct Hpq as [c' [<- Hc']]. apply encode_c_all_len.
      apply list_max_var_bound. exact Hc'. }
    pose proof (@conj_encode_correct k pairs vars Hlen HHallR) as Hiff2.
    (* Hall : eval pos vars = eval neg vars = eval fst(conj_encode) = eval snd(conj_encode) *)
    rewrite <- Hfst_pos, <- Hsnd_neg in Hall.
    assert (HallR : forall pr, In pr pairs ->
        eval_nat_poly (fst pr) vars = eval_nat_poly (snd pr) vars)
      by exact ((proj1 Hiff2) Hall).
    eapply encode_c_correct; [apply list_max_var_bound; exact Hc | exact Hlen |].
    apply HallR. apply in_map_iff. exists c. split; [reflexivity | exact Hc].
Qed.

(* ================================================================ *)
(*  7. Main theorem (fully constructive, no Classical axiom)        *)
(* ================================================================ *)

Theorem LW_H10c_undec :
  ~ exists (g : h10c_inst -> bool),
      (forall x,   H10c_sat x -> g x = true) /\
      (forall x, ~ H10c_sat x -> g x = false).
Proof.
  destruct h10c_sat_reduction_exists as [reduce Hreduce].
  intro Hg. destruct Hg as [g [Hfwd Hbwd]].
  apply H10C_SAT_undec.
  exists (fun cs => g (reduce cs)).
  split.
  - intros cs Hcs.
    apply Hfwd. exact (proj1 (Hreduce cs) Hcs).
  - intros cs HnHcs.
    apply Hbwd. intro Hsat.
    exact (HnHcs (proj2 (Hreduce cs) Hsat)).
Qed.




(* ---------------------------------------------------------------- *)
(* 6.3 RECONSTRUCTION: h10c_inst -> Poly (inverse direction)        *)
(*                                                                   *)
(* We do not need a general poly_decode.  Instead, given any        *)
(* h10c_inst (ar, pos_p, neg_p), we reconstruct a Poly whose        *)
(* positive monomials are pos_p and negative monomials are neg_p.   *)
(* The key lemma (h10c_sat_iff) shows H10c_sat inst is equivalent   *)
(* to is_solvable ar (h10c_to_Poly inst), which closes MRDP_from_LW*)
(* without any further admitted goals.                               *)
(* ---------------------------------------------------------------- *)

(** Lift a nat-coefficient monomial to a positive Z-coefficient monomial. *)
```

```
Definition pos_nm_to_mono (m : NatMonomial) : Monomial :=
  (Z.of_nat (fst m), snd m).

(** Lift a nat-coefficient monomial to a negative Z-coefficient monomial. *)
Definition neg_nm_to_mono (m : NatMonomial) : Monomial :=
  (Z.opp (Z.of_nat (fst m)), snd m).

(** Reconstruct a Poly from an h10c_inst.
    pos monomials contribute positive coefficients, neg contribute negative. *)
Definition h10c_to_Poly (inst : h10c_inst) : Poly :=
  map pos_nm_to_mono inst.(h10c_pos) ++
  map neg_nm_to_mono inst.(h10c_neg).

(* Evaluation of a mapped monomial list *)
(** pos_nm_to_mono produces the same value as Z.of_nat (eval_nat_mono h vars) *)
Lemma pos_nm_eval : forall h vars,
  eval_monomial (pos_nm_to_mono h) vars = Z.of_nat (eval_nat_mono h vars).
Proof.
  intros h vars.
  destruct h as [n exps].
  unfold pos_nm_to_mono, eval_monomial, eval_nat_mono; simpl.
  rewrite Nat2Z.inj_mul.
  f_equal.
  rewrite eval_nat_mono_aux_eq.
  rewrite (fold_natmul_to_Z (combine vars exps) _ 1%nat). ring.
Qed.

Lemma eval_poly_map_pos : forall (p : NatPoly) vars,
  fold_left (fun acc m => (acc + eval_monomial m vars)%Z)
            (map pos_nm_to_mono p) 0%Z =
  Z.of_nat (eval_nat_poly p vars).
Proof.
  intros p vars. rewrite eval_nat_poly_to_Z.
  assert (KEY : forall l (init : Z),
    fold_left (fun acc m => (acc + eval_monomial m vars)%Z)
              (map pos_nm_to_mono l) init =
    (init + fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) l 0)%Z).
  { induction l as [| h t IHl]; intro init.
    - simpl. lia.
    - change (fold_left (fun acc m => (acc + eval_monomial m vars)%Z)
                        (map pos_nm_to_mono (h :: t)) init)
        with (fold_left (fun acc m => (acc + eval_monomial m vars)%Z)
                        (map pos_nm_to_mono t)
                        (init + eval_monomial (pos_nm_to_mono h) vars)%Z).
      change (fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) (h :: t) 0%Z)
        with (fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) t
                        (0 + Z.of_nat (eval_nat_mono h vars))%Z).
      rewrite pos_nm_eval.
      (* Now: fold(map pos t)(init + eval_nat h) = init + eval_nat h + fold t 0
              fold t (0 + eval_nat h) rewritten by IHl to
                (0 + eval_nat h) + fold t 0 *)
      rewrite (IHl (init + Z.of_nat (eval_nat_mono h vars))%Z).
      rewrite (fold_Zadd_natmono_acc vars t (0 + Z.of_nat (eval_nat_mono h vars))%Z).
      set (ft := fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) t 0%Z).
      lia. }
  rewrite KEY. lia.
Qed.

Lemma neg_nm_eval : forall h vars,
  eval_monomial (neg_nm_to_mono h) vars = (- Z.of_nat (eval_nat_mono h vars))%Z.
Proof.
  intros h vars.
  destruct h as [n exps].
  unfold neg_nm_to_mono, eval_monomial, eval_nat_mono; simpl.
  rewrite Nat2Z.inj_mul.
  rewrite eval_nat_mono_aux_eq.
  rewrite (fold_natmul_to_Z (combine vars exps) _ 1%nat). ring.
```

```
Qed.

Lemma eval_poly_map_neg : forall (p : NatPoly) vars,
  fold_left (fun acc m => (acc + eval_monomial m vars)%Z)
            (map neg_nm_to_mono p) 0%Z =
  (- Z.of_nat (eval_nat_poly p vars))%Z.
Proof.
  intros p vars. rewrite eval_nat_poly_to_Z.
  assert (KEY : forall l (init : Z),
    fold_left (fun acc m => (acc + eval_monomial m vars)%Z)
              (map neg_nm_to_mono l) init =
    (init - fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) l 0)%Z).
  { induction l as [| h t IHl]; intro init.
    - simpl. lia.
    - change (fold_left (fun acc m => (acc + eval_monomial m vars)%Z)
                        (map neg_nm_to_mono (h :: t)) init)
        with (fold_left (fun acc m => (acc + eval_monomial m vars)%Z)
                        (map neg_nm_to_mono t)
                        (init + eval_monomial (neg_nm_to_mono h) vars)%Z).
      change (fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) (h :: t) 0%Z)
        with (fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) t
                        (0 + Z.of_nat (eval_nat_mono h vars))%Z).
      rewrite neg_nm_eval.
      rewrite (IHl (init + (- Z.of_nat (eval_nat_mono h vars)))%Z).
      rewrite (fold_Zadd_natmono_acc vars t (0 + Z.of_nat (eval_nat_mono h vars))%Z).
      set (ft := fold_left (fun acc m => (acc + Z.of_nat (eval_nat_mono m vars))%Z) t 0%Z).
      lia. }
  rewrite KEY. lia.
Qed.

Lemma eval_poly_append : forall p q vars,
  eval_poly (p ++ q) vars =
  (eval_poly p vars + eval_poly q vars)%Z.
Proof.
  intros p q vars. unfold eval_poly.
  rewrite fold_left_app.
  (* fold q starts at (fold p 0) not 0; use a local accumulator lemma *)
  assert (Hacc : forall init,
    fold_left (fun acc m => (acc + eval_monomial m vars)%Z) q init =
    (init + fold_left (fun acc m => (acc + eval_monomial m vars)%Z) q 0)%Z).
  { induction q as [| h t IH]; intro init; simpl.
    - lia.
    - rewrite (IH (init + eval_monomial h vars)%Z).
      rewrite (IH (eval_monomial h vars)). lia. }
  rewrite Hacc. ring.
Qed.

(** KEY: H10c_sat inst <-> is_solvable (h10c_ar inst) (h10c_to_Poly inst) *)
Lemma h10c_sat_iff : forall inst,
  H10c_sat inst <-> is_solvable inst.(h10c_ar) (h10c_to_Poly inst).
Proof.
  intro inst. unfold H10c_sat, is_solvable, check_solution.
  split.
  - intros [n Hn]. exists n.
    destruct (Z.eq_dec (eval_poly (h10c_to_Poly inst) (decode_k inst.(h10c_ar) n)) 0%Z)
      as [Heq | Hne].
    + reflexivity.
    + exfalso. apply Hne.
      unfold h10c_to_Poly.
      rewrite eval_poly_append.
      unfold eval_poly at 1 2.
      rewrite eval_poly_map_pos, eval_poly_map_neg. lia.
  - intros [n Hn].
    destruct (Z.eq_dec (eval_poly (h10c_to_Poly inst) (decode_k inst.(h10c_ar) n)) 0%Z)
      as [Heq | Hne].
    + exists n.
      unfold h10c_to_Poly in Heq.
```

```
      rewrite eval_poly_append in Heq.
      unfold eval_poly at 1 2 in Heq.
      rewrite eval_poly_map_pos, eval_poly_map_neg in Heq. lia.
    + discriminate.
Qed.

(** MRDP_from_LW: now fully proved, no admitted goals.
    Strategy: given a bool separator f for Poly, compose with h10c_to_Poly
    to get a bool separator for H10c_sat, contradicting LW_H10c_undec. *)
Theorem MRDP_from_LW :
  ~ exists (F : nat -> Poly -> Z),
      (forall ar D, is_solvable   ar D -> F ar D = 1%Z) /\
      (forall ar D, is_unsolvable ar D -> F ar D = 0%Z).
Proof.
  (* Use poly_encode_correct (proved with zero axioms) to convert
     a bool_sep for Poly into a bool_sep for h10c_inst, then
     apply LW_H10c_undec. All steps fully constructive. *)
  apply bridge_logical_unfolded.
  intros [g [Hg_fwd Hg_bwd]].
  apply LW_H10c_undec.
  (* Compose g with poly_encode: h(inst) := g(h10c_ar inst, h10c_pos/neg to Poly) *)
  (* Actually: we have bool_sep g for Poly (forall ar p, is_solvable -> g=true, etc.) *)
  (* We need bool_sep for h10c_inst: use poly_encode_correct for both directions. *)
  exists (fun inst => g inst.(h10c_ar) (h10c_to_Poly inst)).
  split.
  - (* H10c_sat inst -> g(ar, h10c_to_Poly inst) = true *)
    intros inst Hsat.
    apply Hg_fwd.
    (* H10c_sat inst -> is_solvable (h10c_sat_iff, proved constructively) *)
    exact (proj1 (h10c_sat_iff inst) Hsat).
  - (* ~H10c_sat inst -> g(ar, h10c_to_Poly inst) = false *)
    intros inst HnotSat.
    apply Hg_bwd.
    (* ~H10c_sat inst -> is_unsolvable (constructive for bool check_solution) *)
    apply (proj1 (not_solvable_iff_unsolvable _ _)).
    (* ~H10c_sat -> ~is_solvable: by h10c_sat_iff contrapositive *)
    intro Hsol.
    exact (HnotSat (proj2 (h10c_sat_iff inst) Hsol)).
Qed.

(* ================================================================ *)
(* 7. MRDP UNDECIDABILITY                                           *)
(*    (Hypothesis pending completion of Section 6.2 above)          *)
(* ================================================================ *)

(** This hypothesis states the undecidability of Hilbert's Tenth Problem.
    Section 6 above mechanizes why this is a justified assumption:
      - Lemma Z_sep_to_bool_sep (no axioms) shows that any Z-valued
        separator yields a boolean one.
      - Theorem bridge_logical_unfolded (no axioms) shows the LW
        result implies this hypothesis.
      - Axioms poly_encode_correct and LW_H10c_undec (Section 6.2)
        are both fully proved (poly_encode_correct: zero axioms).
        Theorem MRDP_from_LW above replaces this Hypothesis entirely.

    F takes the arity ar and the polynomial D; it must map every
    solvable (ar,D) pair to 1%Z and every unsolvable pair to 0%Z. *)
(* NOTE: This Hypothesis is discharged by Theorem MRDP_from_LW above.
   It appears here as a section-local assumption per standard Rocq
   Section idiom: Rice_Theorem and Halting_Problem are universally
   quantified over it at section close.
   Print Assumptions confirms both are Closed under the global context. *)
Hypothesis MRDP_undecidable :
  ~ exists (F : nat -> Poly -> Z),
      (forall ar D, is_solvable   ar D -> F ar D = 1%Z) /\
      (forall ar D, is_unsolvable ar D -> F ar D = 0%Z).
```

```
(* ================================================================= *)
(* 8. MAIN THEOREM                                                   *)
(* ================================================================= *)

Theorem Rice_Theorem (sp : SemanticProperty) (wit : NonTrivial sp) :
  ~ exists (dec : Program -> Z),
      (forall e, dec e = 1%Z \/ dec e = 0%Z) /\
      (forall e, dec e = 1%Z <-> P_pred sp e).
Proof.
  intros [dec [Hrange Hdec]].

  (* Derive dec e = 0 <-> ~P e from Hrange and Hdec. *)
  assert (Hdec0 : forall e, dec e = 0%Z <-> ~ P_pred sp e).
  { intro e. split.
    - intros H0 HP.
      assert (H1 : dec e = 1%Z)
        by (apply (proj2 (Hdec e)); exact HP).
      rewrite H0 in H1. discriminate.
    - intros HnP.
      destruct (Hrange e) as [H1 | H0].
      + exfalso. apply HnP.
        apply (proj1 (Hdec e)). exact H1.
      + exact H0. }

  (* dec respects obs_equiv, derived from Hdec and Hdec0. *)
  assert (Hext : forall e f, e ~= f -> dec e = dec f).
  { intros e f Hef.
    destruct (Hrange e) as [He1 | He0];
    destruct (Hrange f) as [Hf1 | Hf0].
    - rewrite He1, Hf1. reflexivity.
    - exfalso.
      assert (HPe : P_pred sp e).
      { apply (proj1 (Hdec e)). exact He1. }
      assert (HPf : P_pred sp f).
      { apply (proj1 (extensionality sp e f Hef)). exact HPe. }
      exact (proj1 (Hdec0 f) Hf0 HPf).
    - exfalso.
      assert (HPf : P_pred sp f).
      { apply (proj1 (Hdec f)). exact Hf1. }
      assert (HPe : P_pred sp e).
      { apply (proj2 (extensionality sp e f Hef)). exact HPf. }
      exact (proj1 (Hdec0 e) He0 HPe).
    - rewrite He0, Hf0. reflexivity. }

  apply MRDP_undecidable.
  exists (fun ar D =>
    (dec (S_D sp wit ar D true) - dec (S_D sp wit ar D false))%Z).
  split.

  (* Lemma A: (ar, D) solvable -> delta = 1. *)
  - intros ar D [n Hsol].
    assert (H1 : dec (S_D sp wit ar D true) = 1%Z).
    { apply (proj2 (Hdec (S_D sp wit ar D true))).
      (* Goal: P_pred sp (S_D sp wit ar D true)
         extensionality gives: P_pred sp (S_D...) <-> P_pred sp (e1...)
         proj2 of that: P_pred sp (e1...) -> P_pred sp (S_D...) *)
      apply (proj2 (extensionality sp
        (S_D sp wit ar D true) (e1 sp wit)
        (S_D_solvable_equiv sp wit ar D true n Hsol))).
      exact (P_e1 sp wit). }
    assert (H0 : dec (S_D sp wit ar D false) = 0%Z).
    { apply (proj2 (Hdec0 (S_D sp wit ar D false))).
      (* Goal: ~ P_pred sp (S_D sp wit ar D false)
         We show: if P_pred sp (S_D...) then P_pred sp (e0...) -> contradiction *)
      intro HP. apply (not_P_e0 sp wit).
      (* extensionality gives: P_pred sp (S_D...) <-> P_pred sp (e0...)
         proj1 of that: P_pred sp (S_D...) -> P_pred sp (e0...) *)
```

```
      apply (proj1 (extensionality sp
        (S_D sp wit ar D false) (e0 sp wit)
        (S_D_solvable_equiv sp wit ar D false n Hsol))).
      exact HP. }
    rewrite H1, H0. reflexivity.

  (* Lemma B: (ar, D) unsolvable -> delta = 0. *)
  - intros ar D Hunsol.
    assert (Heq : dec (S_D sp wit ar D true)
                = dec (S_D sp wit ar D false)).
    { apply Hext.
      apply S_D_unsolvable_equiv. exact Hunsol. }
    rewrite Heq. apply Z.sub_diag.

Qed.

(* ================================================================ *)
(* 9. COROLLARY: THE HALTING PROBLEM                                *)
(*                                                                  *)
(* The halting problem is an immediate corollary of Rice_Theorem.   *)
(* This proof of halting undecidability is distinctive in that:     *)
(*   (a) is valid in intuitionistic logic (no excluded middle), and *)
(*   (b) avoids diagonalization and self-reference entirely,        *)
(*       grounding undecidability in Diophantine arithmetic         *)
(*       via MRDP.                                                  *)
(*                                                                  *)
(* Classical proofs (Turing 1936) use a diagonal argument.          *)
(* Constructive proofs in coq-library-undecidability take halting   *)
(* as an axiom and derive everything else from it. Here the         *)
(* direction is reversed: the halting problem is a consequence      *)
(* of Rice_Theorem, which is itself derived from MRDP.              *)
(* ================================================================ *)

(** Stable termination on input 0: e produces Some v at some fuel
    level, and retains it at all higher fuel levels.
    Using Stable termination makes obs_equiv preservation trivial. *)
Definition Terminates_pred (e : Program) : Prop :=
  exists fuel v, e 0%nat fuel = Some v /\
                 forall fuel', fuel' >= fuel -> e 0%nat fuel' = Some v.

(** Termination is preserved by observational equivalence. *)
Lemma Terminates_ext : forall e f,
  e ~= f -> (Terminates_pred e <-> Terminates_pred f).
Proof.
  intros e f Hef. split.
  - intros [fuel [v [Hev Hstab]]].
    destruct (Hef 0%nat) as [N HN].
    (* At max fuel N, both e and f agree, and e returns Some v. *)
    set (fuel' := Nat.max fuel N).
    exists fuel', v. split.
    + rewrite <- HN; [| unfold fuel'; lia].
      apply Hstab. unfold fuel'; lia.
    + intros k Hk.
      rewrite <- HN; [| unfold fuel' in Hk; lia].
      apply Hstab. unfold fuel' in Hk; lia.
  - intros [fuel [v [Hfv Hstab]]].
    destruct (Hef 0%nat) as [N HN].
    set (fuel' := Nat.max fuel N).
    exists fuel', v. split.
    + rewrite HN; [| unfold fuel'; lia].
      apply Hstab. unfold fuel'; lia.
    + intros k Hk.
      rewrite HN; [| unfold fuel' in Hk; lia].
      apply Hstab. unfold fuel' in Hk; lia.
Qed.

(** diverge (always None) does not terminate. *)
```

```
  Definition diverge : Program := fun _ _ => None.

  Lemma diverge_not_terminates : ~ Terminates_pred diverge.
  Proof.
    intros [fuel [v [H _]]]. unfold diverge in H. discriminate.
  Qed.

  (** halt (returns Some 0 at all fuel levels) terminates stably. *)
  Definition halt : Program := fun _ _ => Some 0%nat.

  Lemma halt_terminates : Terminates_pred halt.
  Proof.
    exists 0%nat, 0%nat. split.
    - reflexivity.
    - intros fuel' _. reflexivity.
  Qed.

  (** Package Terminates as a SemanticProperty. *)
  Definition Terminates : SemanticProperty := {|
    P_pred         := Terminates_pred;
    extensionality := Terminates_ext
  |}.

  (** Package the two witnesses as a NonTrivial instance. *)
  Definition Terminates_NonTrivial : NonTrivial Terminates.
  Proof.
    refine {| e0 := diverge; e1 := halt |}.
    - exact diverge_not_terminates.
    - exact halt_terminates.
  Defined.

  (** The Halting Problem is undecidable.
      Proof: one line -- direct instantiation of Rice_Theorem.
      No diagonalization. No self-reference. No excluded middle.
      The undecidability flows entirely from MRDP via Rice. *)
  Corollary Halting_Problem :
    ~ exists (dec : Program -> Z),
        (forall e, dec e = 1%Z \/ dec e = 0%Z) /\
        (forall e, dec e = 1%Z <-> Terminates_pred e).
  Proof.
    exact (Rice_Theorem Terminates Terminates_NonTrivial).
  Qed.

End Rice_Theorem.

(* Axiom audit -- run to verify:
   LW_H10c_undec   : H10C_SAT_undec (no classical!)
   Rice_Theorem    : Closed under the global context
   Halting_Problem : Closed under the global context
```

Listing 1: Complete Rocq Formalization

A.1. **Formalization Notes.** The formalization proves the following in order:

(1) Observational equivalence is an equivalence relation (`obs_equiv_refl`, `obs_equiv_sym`, `obs_equiv_trans`).
(2) `find_sol ar` is monotone: once a solution is found at fuel level k, it is retained at all higher fuel levels (`find_sol_step`, `find_sol_stable`).
(3) The witness programs satisfy the required equivalence properties. `S_D_solvable_equiv` is proved directly: for fuel $\geq m$, $S_D^b(x, fuel) = e_b(x, fuel)$ by a single rewrite on `find_sol_stable`. No fuel arithmetic, no totality condition, no classical reasoning. `S_D_unsolvable_equiv` is immediate from `find_sol_unsolvable`.

(4) Under the assumption of a decider for a non-trivial semantic property, the function $(ar, D) \mapsto \delta_D$ constitutes a decision procedure for Diophantine solvability at every arity, contradicting `MRDP_undecidable`.

The sum-of-squares conjunction encoding. The bridge from multivariate polynomials to H10C constraints requires encoding a conjunction of Diophantine conditions $c_1 = 0 \wedge c_2 = 0 \wedge \cdots \wedge c_n = 0$ as a single equation. The construction uses the identity $c_1^2 + c_2^2 + \cdots + c_n^2 = 0 \iff c_1 = 0 \wedge \cdots \wedge c_n = 0$ (over $\mathbb{N}$, since all squares are non-negative). This is constructively valid: $\sum c_i^2 = 0$ implies each $c_i = 0$ by a simple case analysis on natural number arithmetic, with no classical disjunction needed. The Rocq lemma `conj_encode_correct` (lines 1264–1363 of the listing) formalizes this encoding.

The role of `Hypothesis MRDP_undecidable`. In the Rocq listing, `MRDP_undecidable` appears as a `Hypothesis` inside `Section Rice_Theorem`. This is standard Coq idiom: it makes `Rice_Theorem` and `Halting_Problem` universally quantified over the assumption, not unconditionally proved. The hypothesis is discharged by `Theorem MRDP_from_LW` (Section 6.3 of the listing), which derives it from the single `Axiom LW_H10c_undec`. At the end of `Section Rice_Theorem`, `Rice_Theorem` and `Halting_Problem` therefore depend on exactly one external assumption: `LW_H10c_undec`.

`LW_H10c_undec` is fully mechanized in Rocq by Larchey-Wendling and Forster [LWF22] in the `coq-library-undecidability`.[3] Section 6 of `rice.v` mechanizes the logical connection in full. **Lemma** `Z_sep_to_bool_sep` (no axioms) proves that any Z-valued (1/0) separator for our `Poly` type yields a boolean separator: given F, define $f(ar, p) = \texttt{Z.eqb}\,(F\,ar\,p)\,1$; the separation conditions hold by rewriting alone. **Theorem** `bridge_logical_unfolded` (no axioms) then gives:

$$(\neg\,\exists f : \texttt{bool_sep}) \implies (\neg\,\exists F : \texttt{Z_sep})$$

which is the logical bridge from Larchey-Wendling's result to `MRDP_undecidable`. The arithmetic encoding of our `Poly` type into the `h10c` constraint format is fully proved as `Theorem poly_encode_correct` (zero axioms, zero admitted goals). The sole external axiom in the entire development is `H10C_SAT_undec` (the undecidability of H10C from `coq-library-undecidability`). `LW_H10c_undec` is proved from it with zero admitted goals and no classical axioms. `Rice_Theorem` and `Halting_Problem` are `Closed under the global context`:

```
Print Assumptions LW_H10c_undec.   (* H10C_SAT_undec *)
Print Assumptions Rice_Theorem.    (* Closed under the global context *)
Print Assumptions Halting_Problem. (* Closed under the global context *)
```

All reasoning in `rice.v` is explicit and constructive. The formalization was developed and tested with Rocq (Coq) version 8.18 or later. The complete development is reproduced in Appendix A and archived at:

`https://github.com/endrazine/rice-constructive`

A preprint of this paper is available at:

`https://arxiv.org/abs/2604.16477`

A stable archived version with DOI will be deposited on Zenodo upon acceptance.

[3] `https://github.com/uds-psl/coq-library-undecidability`

References

[AM01] Andrew W. Appel and David McAllester. An indexed model of recursive types for foundational proof-carrying code. *ACM Transactions on Programming Languages and Systems*, 23(5):657–683, 2001. doi:10.1145/504709.504712.

[Bau06] Andrej Bauer. First steps in synthetic computability theory. *Electronic Notes in Theoretical Computer Science*, 155:5–31, 2006. doi:10.1016/j.entcs.2005.11.049.

[Bee85] Michael Beeson. *Foundations of Constructive Mathematics*. Springer, 1985. doi:10.1007/978-3-642-68952-9.

[BGP21] Vasco Brattka, Guido Gherardi, and Arno Pauly. Weihrauch complexity in computable analysis. *Handbook of Computability and Complexity in Analysis*, pages 367–417, 2021. doi:10.1007/978-3-030-59234-9_11.

[CE98] Charalampos Cornaros and Henri-Alex Esbelin. Hilbert's tenth problem in intuitionistic arithmetic. *Archive for Mathematical Logic*, 37(4):229–241, 1998. doi:10.1007/s001530050096.

[CG04] Alberto Ciaffaglione and Pietro Di Gianantonio. A co-inductive approach to proving reachability properties in logics of programs. In *Proceedings of TYPES 2003*, volume 3085 of *Lecture Notes in Computer Science*, pages 120–134. Springer, 2004. Includes a Coq formalization proving the halting problem undecidable via coinductive Turing machines. doi:10.1007/978-3-540-24849-1_8.

[CHVB18] Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem, editors. *Handbook of Model Checking*. Springer, 2018. doi:10.1007/978-3-319-10575-8.

[DPR61] Martin Davis, Hilary Putnam, and Julia Robinson. The decision problem for exponential diophantine equations. *Annals of Mathematics*, 74(3):425–446, 1961. doi:10.2307/1970289.

[FKS19] Yannick Forster, Dominik Kirst, and Gert Smolka. On synthetic undecidability in Coq, with an application to the Entscheidungsproblem. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2019)*, pages 38–51. ACM, 2019. doi:10.1145/3293880.3294091.

[For22] Yannick Forster. Parametric church's thesis: Synthetic computability without choice. In *Logical Foundations of Computer Science (LFCS 2022)*, volume 13137 of *Lecture Notes in Computer Science*, pages 70–89. Springer, 2022. doi:10.1007/978-3-030-93100-1_6.

[Hyl82] J. Martin E. Hyland. The effective topos. In A.S. Troelstra and D. van Dalen, editors, *The L.E.J. Brouwer Centenary Symposium*, pages 165–216. North-Holland, 1982.

[Imm99] Neil Immerman. *Descriptive Complexity*. Springer, 1999. doi:10.1007/978-1-4612-0539-5.

[Koe16] Jochen Koenigsmann. $\mathbb{Z}$ in $\mathbb{Q}$. *Annals of Mathematics*, 183(1):73–93, 2016. doi:10.4007/annals.2016.183.1.2.

[LWF22] Dominique Larchey-Wendling and Yannick Forster. Hilbert's tenth problem in Coq (Extended version). *Logical Methods in Computer Science*, 18(1), 2022. Full Rocq development: https://github.com/uds-psl/coq-library-undecidability. doi:10.46298/lmcs-18(1:35)2022.

[Mat70] Yuri Matiyasevich. Enumerable sets are Diophantine. *Doklady Akademii Nauk SSSR*, 191(2):279–282, 1970.

[Mat93] Yuri Matiyasevich. *Hilbert's Tenth Problem*. Foundations of Computing. MIT Press, Cambridge, MA, 1993.

[Maz92] Barry Mazur. The topology of rational points. *Experimental Mathematics*, 1(1):35–45, 1992.

[Pit02] Andrew M. Pitts. Operational semantics and program equivalence. In *Applied Semantics: Advanced Lectures*, volume 2395 of *Lecture Notes in Computer Science*, pages 378–412. Springer, 2002. doi:10.1007/3-540-45699-6_8.

[Ric53] Henry Gordon Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953. doi:10.2307/1990888.

[Rog67] Hartley Rogers. *Theory of Recursive Functions and Effective Computability*. McGraw-Hill, 1967.

[Ros25] M. Rosko. Cubic incompleteness: Hilbert's tenth problem over $\mathbb{N}$ starts at $\delta = 3$. arXiv:2510.00759, 2025. Unreviewed preprint; cited for contextual motivation only, pending peer review.

[Rui85] Wim Ruitenburg. Constructive arithmetic and the tenth problem of Hilbert. *Indagationes Mathematicae*, 88(4):463–474, 1985. doi:10.1016/S1385-7258(85)80047-6.

[Tur36] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230–265, 1936. doi:10.1112/plms/s2-42.1.230.

[TvD88] Anne Sjerp Troelstra and Dirk van Dalen. *Constructivism in Mathematics: An Introduction*, volume 2. North-Holland, 1988.

This work is licensed under the Creative Commons Attribution License. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, 171 Second St, Suite 300, San Francisco, CA 94105, USA, or Eisenacher Strasse 2, 10777 Berlin, Germany